Deep Learning: Basic Techniques

Study Notes – Working Draft

Stefan Eder

Based on the lecture *Deep Learning: Basic Techniques* (G. Klambauer, JKU Linz), slides of WS 2025/26

These notes walk through the basic techniques of deep learning the way the lecture builds them up: from a single linear neuron, via the perceptron, logistic and softmax regression, to multi-layer and deep networks trained with backpropagation. Every idea comes in plain words first, then as a formula, and wherever it helps there is a small example with actual numbers. All worked numbers were re-computed by script, and the backpropagation example was checked against finite differences. Use the notes as a study aid alongside the lecture slides, not as a replacement for them.

Working draft: Chapters 1, 3 and 4 are written out. Chapter 2 and Chapters 5–10 are still outlines and will be filled in once their slides are available.

If you spot an error or a gap, please open an issue or pull request at **github.com/steveTgit/AI_Master** — corrections are always welcome.

Good study materials should be shared, not gatekept. If this helped you, take it as a reason to help the next colleague you see struggling with something.

October 20, 2026

Contents

1 A Brief History of Neural Networks and Deep Learning . . . 1
- 1.1 About the Course . . . 1
- 1.2 Machine Learning and Deep Learning . . . 2
- 1.3 A Brief History . . . 2
  - 1.3.1 Before 1940: Least Squares, Gradient Descent and the Chain Rule . . . 3
  - 1.3.2 1943–1969: The First Neurons, Hebb and the Perceptron . . . 3
  - 1.3.3 1974–1989: Backpropagation and Multi-Layer Perceptrons . . . 4
  - 1.3.4 1991–1997: The Vanishing Gradient Problem and LSTM . . . 4
  - 1.3.5 Around 2010: Re-Emergence as Deep Learning . . . 4
- 1.4 Why Deep Learning Took Off . . . 5
- 1.5 Main Types of Networks . . . 6
- 1.6 Credit, Prizes and Disputes . . . 6
- 1.7 Neural-Network History versus AI History . . . 6
- 1.8 Chapter Checklist . . . 7

2 Biological Neural Networks . . . 8
- 2.1 The Biological Neuron . . . 8
- 2.2 Learning in Biological Networks . . . 8
- 2.3 From Biological to Artificial Neurons . . . 8
- 2.4 Chapter Checklist . . . 8

3 Notation . . . 9
- 3.1 Data . . . 9
- 3.2 Models, Losses and Learning . . . 9
- 3.3 Units and Weights . . . 10
- 3.4 Layer-Wise Notation (Deep Networks) . . . 10
- 3.5 Further Conventions . . . 11

4 Basic Neural Network Architectures . . . 12
- 4.1 Linear Model: The Linear Neuron . . . 13
  - 4.1.1 From Gaussian Noise to the Squared Loss . . . 13
  - 4.1.2 Learning: Empirical Risk Minimization . . . 14
  - 4.1.3 Closed-Form Solution: The Normal Equations . . . 14
  - 4.1.4 Gradient Descent and Stochastic Gradient Descent . . . 15
  - 4.1.5 The Trained Network . . . 15
- 4.2 Perceptron . . . 16
  - 4.2.1 Perceptron Loss and Learning Rule . . . 16
  - 4.2.2 Geometric Interpretation . . . 17
  - 4.2.3 Convergence for Linearly Separable Data . . . 17
- 4.3 Linear Neuron with Sigmoid Activation: Logistic Regression . . . 18
  - 4.3.1 Likelihood and Binary Cross-Entropy . . . 19
  - 4.3.2 Gradient and Hessian . . . 19
  - 4.3.3 Learning . . . 20

4.3.4 Intermezzo: Optimization and Newton's Method . . . 21
4.4 Linear Neuron with Softmax Activation: Softmax Regression . . . 22
4.4.1 Categorical Cross-Entropy . . . 22
4.4.2 Gradient . . . 23
4.4.3 Comparing the Single-Layer Networks . . . 24
4.5 Single-Layer Networks Cannot Solve XOR . . . 24
4.5.1 Why a Linear Network Fails . . . 24
4.5.2 A Hidden Layer Solves It . . . 25
4.6 Multi-Layer Perceptron . . . 26
4.6.1 Model and Forward Pass . . . 26
4.6.2 Gradient Descent and the Backpropagation Algorithm . . . 28
4.6.3 Training MLPs . . . 31
4.7 (Deep) Feed-Forward Neural Networks . . . 32
4.7.1 Forward Pass . . . 32
4.7.2 Backpropagation in DNNs . . . 32
4.7.3 The Vanishing Gradient Problem . . . 34
4.8 Jacobian . . . 35
4.9 Hessian . . . 36
4.10 Efficient Training: Basic Tricks of the Trade . . . 36
4.10.1 Training, Validation and Test Set . . . 36
4.10.2 Online, Stochastic and Batch Training . . . 37
4.10.3 Epochs, Batch Size and Learning Curves . . . 37
4.10.4 Normalizing Inputs and Outputs . . . 38
4.10.5 Initialization . . . 38
4.10.6 Learning Rates . . . 38
4.10.7 Number of Neurons and Hidden Layers . . . 38
4.11 Excursion: Amari–Hopfield Networks . . . 39
4.12 Chapter Checklist . . . 40

5 Generalization Error, Test Set and Cross-Validation . . . 41
5.1 Empirical Error and Generalization Error . . . 41
5.2 Overfitting and Underfitting . . . 41
5.3 Estimating the Generalization Error: Test Set . . . 41
5.4 Cross-Validation . . . 41
5.5 Chapter Checklist . . . 41

6 Convolutional Neural Networks . . . 42
6.1 Convolution and Cross-Correlation . . . 42
6.2 Convolutional Layers . . . 42
6.3 Pooling . . . 42
6.4 CNN Architectures and Training . . . 42
6.5 Chapter Checklist . . . 42

7 Learning Methods: Basic Algorithms . . . 43
7.1 Gradient Descent Revisited . . . 43
7.2 Momentum and Adaptive Learning Rates . . . 43
7.3 Learning-Rate Schedules . . . 43
7.4 Second-Order Methods . . . 43
7.5 Chapter Checklist . . . 43

8 Regularization . . . 44

8.1 Weight Decay (L_2 and L_1 Penalties) . . . 44
8.2 Early Stopping . . . 44
8.3 Dropout and Other Noise-Based Methods . . . 44
8.4 Chapter Checklist . . . 44

9 Initialization . . . 45
9.1 Why Initialization Matters . . . 45
9.2 Variance-Preserving Initialization . . . 45
9.3 Chapter Checklist . . . 45

10 Normalization . . . 46
10.1 Input Normalization Revisited . . . 46
10.2 Batch Normalization and Layer Normalization . . . 46
10.3 Self-Normalizing Networks . . . 46
10.4 Chapter Checklist . . . 46

1

A Brief History of Neural Networks and Deep Learning

The first lecture does two things. It sets up the course (what it covers, how it is examined, what to read), and it tells the story of how neural networks went from a 1940s model of a nerve cell to the technology behind almost every recent AI headline. The history is worth knowing for its own sake, but it also previews the technical content: nearly every milestone in this chapter — the perceptron, the XOR problem, backpropagation, the vanishing gradient, ReLUs, convolutions — comes back later in these notes as a formula.

1.1 About the Course

Deep Learning: Basic Techniques (some slide decks use the title *Deep Learning and Neural Networks I*) is a first-semester basic course of the AI master at JKU Linz. It consists of a lecture (VO, 2 hours per week, 3 ECTS), examined by a written on-site exam at the end of the semester, and an exercise (UE, 1 hour per week, 1.5 ECTS) with five to six roughly bi-weekly assignments. The lecture notes on KUSSS/Moodle contain a reference list; the main textbooks recommended in the first lecture are:

- C. M. Bishop & H. Bishop (2023), *Deep Learning: Foundations and Concepts*, Springer;
- C. M. Bishop (2006), *Pattern Recognition and Machine Learning*, Springer;
- I. Goodfellow, Y. Bengio & A. Courville (2016), *Deep Learning*, MIT Press;
- M. P. Deisenroth, A. A. Faisal & C. S. Ong (2020), *Mathematics for Machine Learning*, Cambridge University Press;
- A. Zhang, Z. C. Lipton, M. Li & A. J. Smola, *Dive into Deep Learning* (freely available online).

Table 1.1 lists the chapters of the course. These notes follow the same numbering, so “Chapter 4” on the slides is Chapter 4 here, and the section numbers 4.1–4.10 match as well.

And one piece of advice from the first lecture that is not about maths: connect with your colleagues. Most of this material is much easier to learn in a group than alone.

Ch.	Topic	Status in these notes
1	A brief history of neural networks	written
2	Biological neural networks	outline
3	Notation	written (collected from Ch. 4)
4	Basic neural network architectures (4.1–4.10)	written
5	Generalization error, test set and cross-validation	outline
6	Convolutional neural networks	outline
7	Learning methods: basic algorithms	outline
8	Regularization	outline
9	Initialization	outline
10	Normalization	outline

Table 1.1: Roadmap of the course as announced in the first lecture. Chapter 4 is by far the largest: it goes from the single linear neuron (4.1) to deep feed-forward networks, backpropagation and the practical "tricks of the trade" (4.10).

1.2 Machine Learning and Deep Learning

The lecture opens with three nested circles: *intelligent machines and programs* (artificial intelligence), inside it the *ability to learn* (machine learning), and inside that *neural networks* (deep learning). The claim behind the picture is that the current success of AI is, to a large extent, the success of deep learning.

Machine learning and deep learning

Machine learning is "the science of getting computers to act without being explicitly programmed" (Andrew Ng): the system infers knowledge from data and has to *generalize* to data it has not seen, for example in supervised learning, where it learns to classify from labelled examples.

Deep learning is machine learning with (deep) neural networks. Instead of relying on hand-crafted features, deep networks *learn representations* of the data themselves, in several levels: lower levels code simple patterns, higher levels code increasingly abstract concepts that help to solve the task.

Intuition & Mental Model

The standard picture is a face-recognition network. The first layer learns something like edge and blob detectors, the next layers combine them into parts (eyes, noses, mouths), and the top layers respond to whole faces. Nobody programmed "look for an eye" — the hierarchy emerges because it is useful for minimizing the training loss. That is what "learning increasingly abstract features" means, and it is the main difference to classical machine learning, where a human designs the features and the model only learns the last step.

1.3 A Brief History

The history in the lecture follows J. Schmidhuber's review *Deep learning in neural networks: an overview* (Neural Networks, 2015), and the lecture is explicit about a caveat: all history is biased, incomplete and inaccurate — and the history of neural networks is interwoven with, but not the same as, the history of AI. Figure 1.1 gives the overview; the subsections fill in the details.

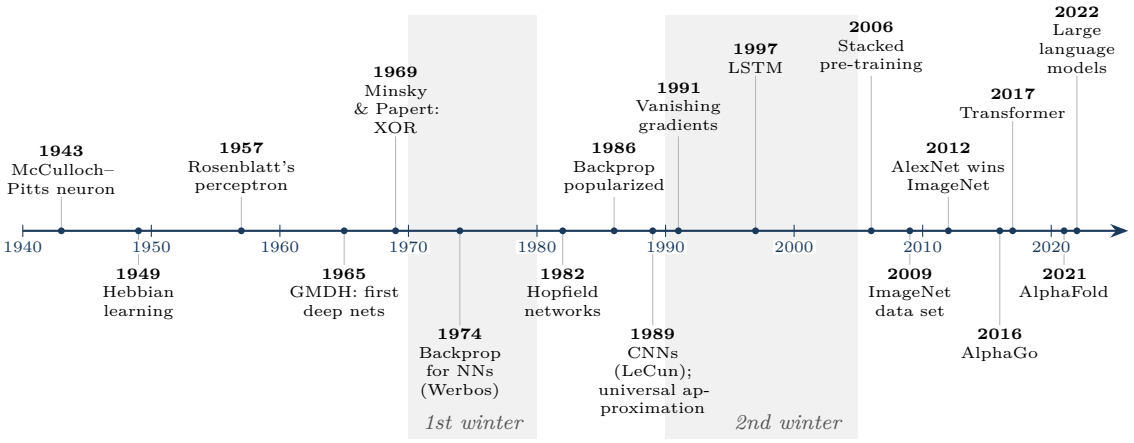


Figure 1.1: Milestones of neural-network history as told in the lecture. The shaded bands are the two "AI winters" for neural networks (1970–∼1980 and 1990–∼2005), periods in which funding and interest dried up. Notice that backpropagation and LSTM were both invented *during* a winter.

1.3.1 Before 1940: Least Squares, Gradient Descent and the Chain Rule

Three ingredients of modern deep learning are centuries old:

- **Linear regression and least squares** (Legendre 1805, Gauss 1809). A linear regression model is exactly a single linear "neuron" (Section 4.1).
- **Gradient descent** (Cauchy 1847, Hadamard 1908): improve a guess by stepping against the gradient.
- **The chain rule** (Leibniz 1676, later Euler): the only mathematical tool behind backpropagation (Section 4.6.2).

Put together, "a linear model fitted by gradient descent on a squared error, with gradients computed by the chain rule" already describes a good part of Chapter 4.

1.3.2 1943–1969: The First Neurons, Hebb and the Perceptron

In 1943, **McCulloch and Pitts** published *A logical calculus of the ideas immanent in nervous activity*, the first computational model of a neuron. Their neuron is a basic logic unit: it takes inputs, forms the weighted sum (dot product of inputs and weights), applies an activation function (a threshold) and "fires" an output. There is no learning — the weights are set by hand.

In 1949, **Donald O. Hebb** proposed a hypothesis about how biological networks learn: the connection between two neurons that fire together is strengthened ("cells that fire together wire together"). This *Hebbian learning* reappears in the Hopfield networks at the end of Chapter 4.

In 1957, **Frank Rosenblatt** built the *perceptron*: an electronic device with adaptable connections, 400 photocells as input neurons, used for image recognition — and, crucially, with the ability to *learn* its weights from examples. Its learning rule is the subject of Section 4.2.

Multi-layer networks are older than often assumed: **Oleksiy Ivakhnenko**'s *Group Method of Data Handling* (GMDH, 1965 and 1971) already trained networks with several layers.

In 1969, **Marvin Minsky and Seymour Papert** published the book *Perceptrons*. Among other things, it showed that a single-layer perceptron cannot solve the simple XOR problem (Section 4.5). The book contributed to a strong reduction of research on neural networks — the

first AI winter (1970–∼1980). Minsky received the Turing Award in 1969.

1.3.3 1974–1989: Backpropagation and Multi-Layer Perceptrons

The way out of the XOR dead end is a hidden layer, and the way to train a hidden layer is backpropagation. **Paul Werbos** suggested backpropagation for neural networks in 1974; it became widely known through **Rumelhart, Hinton and Williams** (1986) in the *Parallel Distributed Processing* volumes, which showed how to train multi-layer perceptrons (MLPs) with it.

In 1989, **Cybenko** and **Kurt Hornik** proved versions of the *universal function approximation theorem*: an MLP with a single hidden layer and enough hidden units can approximate any continuous function on a compact set arbitrarily well. So MLPs do not only solve XOR — they can, in principle, represent (almost) anything.

Two more architectures appear in this period. In 1982, **John Hopfield** introduced *Hopfield networks*, recurrent networks that work as associative memories. And convolutional networks have their roots here too: **Kunihiko Fukushima**'s *Neocognitron* (1979) introduced the architecture (but was not trained by backprop), and **Yann LeCun** trained convolutional neural networks with backpropagation for image recognition in 1989.

The 2024 Nobel Prize in Physics went to Hopfield ("Hopfield networks") and Hinton ("Boltzmann machines") for foundational work on neural networks.

1.3.4 1991–1997: The Vanishing Gradient Problem and LSTM

The vanishing gradient problem (VGP)

Training a neural network relies on gradients, which are propagated backward through the layers. In deep networks (and in recurrent networks unrolled over time) these gradients shrink — or blow up — exponentially with the number of layers they pass. The deeper the network, the harder it is to train. Discovered by **Sepp Hochreiter** in his 1991 diploma thesis; called by the lecture "the fundamental problem of all deep learning methods".

The mathematical reason is derived in Section 4.7.3. For recurrent networks, Hochreiter and Schmidhuber's **Long Short-Term Memory (LSTM)** (1997) ameliorates the problem; for feed-forward networks, ReLUs, careful initialization and skip connections help (Section 4.7.3).

The 1990s and early 2000s were the **second AI winter** for neural networks (1990–∼2005): support-vector machines and other kernel methods dominated machine learning, and neural networks were considered hard to train.

1.3.5 Around 2010: Re-Emergence as Deep Learning

Deep learning took off when several developments came together:

- **GPU training** of neural networks (Oh and Jung, 2004);
- **stacked, layer-wise training** of deep networks (Hinton et al., 2006);
- **rectified linear units**, ReLUs (Nair and Hinton, 2010);
- **deep CNNs** trained on GPUs (Krizhevsky et al., 2012);
- **large data sets**: in 2009, Fei-Fei Li launched *ImageNet*, a database of about 14 million labelled images that became *the* benchmark of deep learning research.

The lecture dates the starting point of the deep learning era to about 2010. In 2018, Yoshua Bengio, Geoffrey Hinton and Yann LeCun received the Turing Award for deep learning.

1.4 Why Deep Learning Took Off

Intuition & Mental Model

Neural networks are not new; what changed around 2010–2012 is the environment they run in. The lecture lists four reasons for the boom: **fast computers**, **GPUs**, **large data sets** and **algorithmic improvements**. A network that was hopeless to train on a 1995 workstation with a few thousand examples becomes practical with a GPU and a million labelled images — and once it worked, people found the algorithmic tricks (ReLU, better initialization, dropout, . . .) to make it work even better.

The "third wave" started with a series of records around 2012, in vision and in speech:

- **2009:** deep (LSTM-based recurrent) networks won international handwriting-recognition competitions (French, Arabic and Chinese handwriting).
- **2011:** deep networks won the German traffic-sign recognition contest with super-human performance: 0.56 % error, compared with 1.16 % for humans and 3.86 % for the best non-neural competitor.
- **2011:** on an industry speech-recognition benchmark, deep networks lowered the error rate from about 24 % to about 16 % — after the community had been stuck at 24 % for more than a decade. This is when IT companies became seriously interested.
- **2012:** deep networks won the brain-image segmentation challenge with super-human pixel error rate.
- **2012:** a deep CNN trained on GPUs ("AlexNet") won the ImageNet competition with about 16 % error on 1.2 million images, against 26 % for the runner-up.[1]
- **2013:** deep networks won the grand challenge on mitosis detection, and Google researchers improved previous face-recognition results by 70 % using 16,000 CPU cores.

Since then the list of milestones has kept growing. Examples from the lecture:

- **2016:** AlphaGo beats a human Go champion; deep learning systems understand human speech well enough for everyday assistants.
- **2017:** deep networks classify skin cancer at the level of dermatologists (Esteva et al., *Nature*).
- **2021:** AlphaFold, a Transformer-based network for protein structure prediction, is named scientific breakthrough of the year.
- **2022:** deep reinforcement learning steers the magnetic confinement of plasma in a tokamak fusion reactor (Degrave et al., *Nature*).
- **2022:** large language models produce human-like text — the era of LLMs and of the broad uptake of deep learning methods begins.

[1] The record list on the slides dates this to 2013; the winning network was presented at the 2012 ImageNet challenge (Krizhevsky et al., 2012), which is also the date the slides use elsewhere.

1.5 Main Types of Networks

The lecture distinguishes four main families of networks. The useful way to remember them is by the *kind of input* they are built for.

Network type	Input	Typical applications
Feed-forward / fully connected networks	vectors	standard classification problems: spam vs. non-spam, cancer vs. normal tissue
Recurrent neural networks (RNNs)	time series, sequences	text, speech, video; e.g. chatbots
Convolutional neural networks (CNNs)	spatial data (images)	image processing, image recognition
Transformer networks (2017)	sets (of tokens)	natural language processing

Table 1.2: The four main network families. This course covers fully connected networks (Chapter 4) and CNNs (Chapter 6); RNNs, LSTM and Transformers are the topic of later courses.

1.6 Credit, Prizes and Disputes

The history of neural networks is not a settled story, and the lecture shows an example of an ongoing dispute. In 2020, Geoffrey Hinton received the Honda Prize; Jürgen Schmidhuber publicly criticized the choice, arguing that inventions were attributed to Hinton that were not his. He made similar criticism of the 2018 Turing Award to Bengio, Hinton and LeCun, and of the fact that Hopfield networks are usually credited to Hopfield alone, although Shun-Ichi Amari had described the same kind of network in 1972 (this is why the lecture says *Amari–Hopfield* networks, Section 4.11).

A more neutral record is the list of IEEE Neural Networks Pioneer Award winners, which contains most of the names in this chapter — among them Widrow, Kohonen and Grossberg (1991), Amari (1992), Werbos (1995), Hopfield (1997), Hinton (1998), Fukushima (2003), Vapnik (2010), LeCun (2014), Schmidhuber (2016), Bengio (2019) and Hochreiter (2020).

1.7 Neural-Network History versus AI History

Everything above is a history of *neural networks*, not of *AI*. A history of AI would start with Alan Turing and the Turing test, and with the 1956 Dartmouth workshop (J. McCarthy, C. Shannon, R. Solomonoff, T. More, N. Rochester, O. Selfridge, A. Samuel and others), where the term "artificial intelligence" was coined. It would talk about symbolic reasoning, expert systems, and IBM's Deep Blue beating Kasparov at chess in 1997. Broadly, AI went through three waves — the birth of AI in the 1950s and 1960s, expert systems and knowledge engineering in the 1970s and 1980s, and machine learning with deep networks since about 2010 — separated by two winters.

Two different sets of "AI winter" dates

The general AI history places the winters roughly in the mid-1970s and in the late 1980s to early 1990s, triggered by limited computing power and by the failure of expert systems to scale. The neural-network winters on the slides (1970–∼1980 and 1990–∼2005) are shifted, because neural networks fell out of favour within AI at different times. If an exam asks for dates, check which history the question means.

1.8 Chapter Checklist

What you should take away from this chapter:

- what machine learning and deep learning are, and that learning *representations* (increasingly abstract features) is what makes deep learning different;
- the main milestones and their order: McCulloch–Pitts (1943), Hebb (1949), perceptron (1957), Minsky & Papert and XOR (1969), backprop (Werbos 1974; Rumelhart et al. 1986), universal approximation (1989), Hopfield networks (1982), vanishing gradients (1991), LSTM (1997), CNNs (Fukushima 1979; LeCun 1989), ImageNet (2009), the deep learning boom around 2010–2012;
- the two AI winters for neural networks and why they ended;
- the four reasons for the success since 2012: fast computers, GPUs, large data sets, algorithmic improvements;
- the four main network types and the kind of input each is made for.

2

Biological Neural Networks

Artificial neural networks borrowed their name and their basic building block from the brain. This chapter looks at the original: how a biological neuron works, how neurons communicate, and which of these features survive in the artificial neuron of Chapter 4 — and which do not.

Planned content: The slides of this chapter are not yet in the course folder. The section list below is the expected structure; adapt it to the slides once they are available.

2.1 The Biological Neuron

Planned content: Cell body, dendrites, axon and synapses; resting potential and action potential ("spike"); excitatory and inhibitory synapses; synaptic strength as the biological counterpart of a weight.

2.2 Learning in Biological Networks

Planned content: Synaptic plasticity; Hebb's rule ("cells that fire together wire together"); connection to Hebbian learning in Amari–Hopfield networks (Section 4.11).

2.3 From Biological to Artificial Neurons

Planned content: The abstraction: weighted sum of inputs, activation function, output; McCulloch–Pitts neuron; what is lost (spike timing, dendritic computation, energy efficiency) and why the analogy should not be taken too literally.

2.4 Chapter Checklist

Planned content: To be written together with the chapter.

3

Notation

Deep learning papers, textbooks and lectures all use slightly different symbols, and a lot of confusion in exams comes from mixing them up. This chapter collects the notation *as it is used on the Chapter 4 slides* of this lecture, so that the rest of these notes reads consistently. The lecture has its own notation chapter; once its slides are available, this chapter should be checked against them.

3.1 Data

In supervised learning we are given N examples (*samples*, data points), each consisting of an input vector with D *features* and a label. The sample index is written as a **superscript**, so that subscripts stay free for feature (or unit) indices:

$$\boldsymbol{x}^n \in \mathbb{R}^D \quad \text{(input of sample } n\text{)}, \qquad y^n \quad \text{(its label)}, \qquad n = 1, \dots, N.$$

The inputs are collected in the *data matrix* $\boldsymbol{X} = (\boldsymbol{x}^1, \dots, \boldsymbol{x}^N)$, where the lecture's convention is that **samples are rows** and **features are columns**, so $\boldsymbol{X} \in \mathbb{R}^{N \times D}$ and x_{nd} is feature d of sample n. The slides' example: in a data table whose second row is $\boldsymbol{x}^2 = (0.15, 0.14)$, the single entry $x_{21} = 0.15$ is the first feature of the second sample; $\boldsymbol{x}_{\cdot 1}$ is the column of all values of feature 1. Scalar labels are collected in $\boldsymbol{y} = (y^1, \dots, y^N)^\mathsf{T}$.

For classification with K classes, a label is written as a *one-hot vector* $\boldsymbol{y} \in \{0, 1\}^K$ (Section 4.4), and the label vectors of all samples form the label matrix $\boldsymbol{Y}$; y_k^n (or y_{nk}) is entry k of the label of sample n.

3.2 Models, Losses and Learning

- $g(\boldsymbol{x}; \boldsymbol{w})$ — the network (model) as a function of the input $\boldsymbol{x}$, with all its parameters collected in $\boldsymbol{w}$ (or a weight matrix $\boldsymbol{W}$, or several weight matrices $\boldsymbol{W}^{[1]}, \dots, \boldsymbol{W}^{[L]}$). A prediction is $\hat{y} = g(\boldsymbol{x}; \boldsymbol{w})$.
- $\mathcal{L}(y, g(\boldsymbol{x}; \boldsymbol{w}))$ — the **loss function** for a single sample: how bad is the prediction $g(\boldsymbol{x}; \boldsymbol{w})$ if the true label is y?
- $R_{\text{emp}}(\boldsymbol{w}, \boldsymbol{X}, \boldsymbol{y})$ — the **empirical error** (empirical risk): the loss summed or averaged over the training set, $R_{\text{emp}} = \frac{1}{N} \sum_n \mathcal{L}(y^n, g(\boldsymbol{x}^n; \boldsymbol{w}))$. Training means minimizing R_{emp} with respect to $\boldsymbol{w}$.
- η — the **learning rate** (step size) of gradient descent: $\boldsymbol{w}^{\text{new}} = \boldsymbol{w}^{\text{old}} - \eta \nabla_{\boldsymbol{w}} R_{\text{emp}}$.

Loss $\mathcal{L}$ versus number of layers L

The slides use the same letter L for the loss function $L(y, g(\boldsymbol{x}; \boldsymbol{w}))$ and for the number of layers L (and L_ν for the set of units in layer ν). These notes write the loss as $\mathcal{L}$ (calligraphic) to keep the two apart. When reading the slides, the context always tells which one is meant.

3.3 Units and Weights

A neural network consists of *units* (artificial neurons) connected by weighted, directed connections.

Unit-wise notation (single neurons, MLPs)

- w_{ij} — the **weight** (connection strength, synaptic weight) **from unit j to unit i**. Note the order: the *target* comes first.
- $s_i = \sum_j w_{ij} a_j + b_i$ — the **network input** or **pre-activation** of unit i; b_i is its bias. Alternatively the bias is treated as a weight w_{i0} from a constant unit $a_0 = 1$, and the sum runs from $j = 0$.
- f — the **activation function** (also *transfer function*); it may differ between units.
- $a_i = f(s_i)$ — the **activation** (activity, output) of unit i. For input units, $a_i = x_i$.
- Units are numbered consecutively through the whole network: the D input units are $1, \ldots, D$, and if the network has Q units and K outputs, the output units are $Q - K + 1, \ldots, Q$.
- $\delta_i = \frac{\partial \mathcal{L}}{\partial s_i}$ — the **delta error** of unit i (Section 4.6.2).

3.4 Layer-Wise Notation (Deep Networks)

For deep networks it is easier to number the *layers* and to write everything as vectors and matrices. Layers are indexed by a superscript in square brackets.

Layer-wise notation (deep feed-forward networks)

- $\boldsymbol{a}^{[0]} = \boldsymbol{x}$ — the input is the "activation of layer 0".
- $\boldsymbol{W}^{[l]}$ — the weight matrix connecting layer $l-1$ to layer l; entry $w_{ij}^{[l]}$ goes from unit j in layer $l-1$ to unit i in layer l. If layer l has n_l units, $\boldsymbol{W}^{[l]} \in \mathbb{R}^{n_l \times n_{l-1}}$. Bias vector: $\boldsymbol{b}^{[l]}$.
- $\boldsymbol{s}^{[l]} = \boldsymbol{W}^{[l]} \boldsymbol{a}^{[l-1]} + \boldsymbol{b}^{[l]}$ — the pre-activations of layer l.
- $\boldsymbol{a}^{[l]} = f(\boldsymbol{s}^{[l]})$ — the activations of layer l, with f applied element-wise.
- $\hat{\boldsymbol{y}} = \boldsymbol{a}^{[L]}$ — the output of the last layer L (the output activation is often written σ to stress that it may be different from the hidden-layer f).
- $\boldsymbol{\delta}^{[l]} = (\frac{\partial \mathcal{L}}{\partial \boldsymbol{s}^{[l]}})^\mathsf{T}$ — the delta errors of layer l (Section 4.7.2).

Quantity	Unit-wise (MLP)	Layer-wise (DNN)
input	$a_i = x_i$, $i \leq D$	$\boldsymbol{a}^{[0]} = \boldsymbol{x}$
weight	w_{ij} (from j to i)	$\boldsymbol{W}^{[l]}$, entries $w_{ij}^{[l]}$
pre-activation	$s_i = \sum_j w_{ij} a_j + b_i$	$\boldsymbol{s}^{[l]} = \boldsymbol{W}^{[l]} \boldsymbol{a}^{[l-1]} + \boldsymbol{b}^{[l]}$
activation	$a_i = f(s_i)$	$\boldsymbol{a}^{[l]} = f(\boldsymbol{s}^{[l]})$
output	a_k, $Q - K + 1 \leq k \leq Q$	$\hat{\boldsymbol{y}} = \boldsymbol{a}^{[L]}$
delta error	$\delta_i = \partial \mathcal{L} / \partial s_i$	$\boldsymbol{\delta}^{[l]} = (\partial \mathcal{L} / \partial \boldsymbol{s}^{[l]})^\mathsf{T}$

Table 3.1: The two notations side by side. The unit-wise notation is used for single-layer networks and MLPs (Sections 4.1–4.6), the layer-wise notation for deep feed-forward networks (Section 4.7).

3.5 Further Conventions

- Vectors are column vectors and written in bold italic ($\boldsymbol{x}$, $\boldsymbol{w}$); matrices are bold capitals ($\boldsymbol{W}$, $\boldsymbol{X}$). Transposition: $\boldsymbol{w}^\mathsf{T}$.
- $\sigma(x) = 1/(1 + e^{-x})$ is the logistic sigmoid; softmax is defined in Section 4.4.
- $\nabla_{\boldsymbol{w}} R_{\text{emp}}$ is the gradient (a vector of the same shape as $\boldsymbol{w}$); $\nabla^2_{\boldsymbol{w}} R_{\text{emp}}$ the Hessian (Section 4.9).
- $\text{diag}(\boldsymbol{v})$ is the diagonal matrix with $\boldsymbol{v}$ on the diagonal; $\odot$ is the element-wise product.

4

Basic Neural Network Architectures

This is the central chapter of the course. It starts with the simplest possible network — a single neuron that computes a weighted sum — and adds one idea at a time: a threshold (perceptron), a sigmoid (logistic regression), a softmax (multi-class classification), a hidden layer (MLP), many hidden layers (deep networks). Along the way it develops the two tools that every later chapter relies on: *maximum likelihood* to decide which loss to minimize, and *gradient descent with backpropagation* to minimize it.

The section numbers follow the lecture: Sections 4.1–4.4 cover *single-layer* networks, 4.5 shows why they are not enough, 4.6–4.7 cover multi-layer and deep networks with backpropagation, 4.8–4.9 the Jacobian and the Hessian, and 4.10 the practical tricks of the trade. An excursion to Amari–Hopfield networks, which closed the single-layer lecture, comes at the end (Section 4.11).

Units, weights and layers. An artificial neural network is a set of *units* connected by directed, weighted connections. The weight w_{ij} is the connection strength ("synaptic weight") from unit j to unit i. Units are organized in *layers*: an *input* layer that receives the data, one or more *hidden* layers, and an *output* layer (Fig. 4.1). Learning means adapting the weights so that the network computes something useful. The notation is collected in Chapter 3.

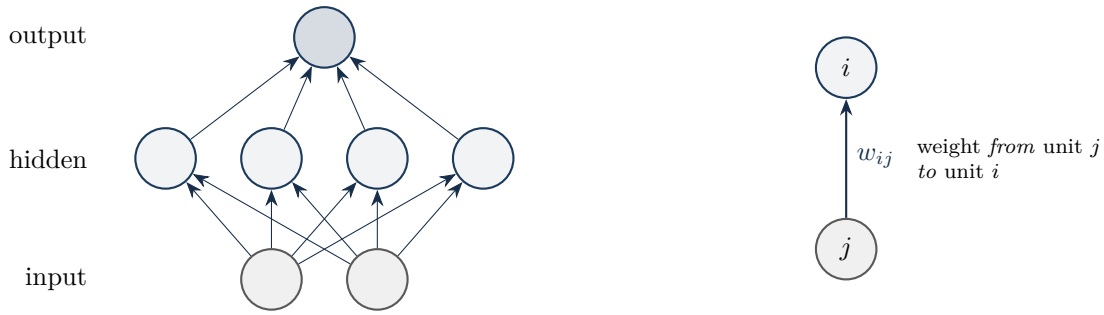


Figure 4.1: Left: a three-layer network with an input, a hidden and an output layer. Right: the weight w_{ij} connects unit j to unit i — the *target* index comes first.

Data for supervised learning. Throughout the chapter we have a supervised data set of N samples. In *regression* the labels are real numbers, $y^n \in \mathbb{R}$; in *classification* they are class labels. The inputs form the data matrix $\boldsymbol{X} \in \mathbb{R}^{N \times D}$ (one sample $\boldsymbol{x}^n$ per row), and the labels form the vector $\boldsymbol{y} = (y^1, \ldots, y^N)^\mathsf{T}$.

4.1 Linear Model: The Linear Neuron

The simplest network has no hidden layer at all: the inputs are directly connected to a single output unit, which outputs the weighted sum of its inputs.

Linear neuron

For an input $\boldsymbol{x} \in \mathbb{R}^D$ and adaptive weights $\boldsymbol{w} \in \mathbb{R}^D$, the linear neuron computes the activation

$$g(\boldsymbol{x}; \boldsymbol{w}) = a = \boldsymbol{w}^\mathsf{T}\boldsymbol{x} = \sum_{d=1}^{D} w_d x_d \; (+\, w_0).$$

A bias w_0 can be included by appending a constant feature $x_0 = 1$ to every input, so we can always write $\boldsymbol{w}^\mathsf{T}\boldsymbol{x}$. The linear neuron is exactly the model of **linear regression**.

4.1.1 From Gaussian Noise to the Squared Loss

Which weights are "good"? The lecture answers this with a probabilistic model, and it is worth understanding the argument, because the same recipe gives us the loss functions for every other network in this chapter.

Assume the label is a deterministic function of the input plus Gaussian noise:

$$y = g(\boldsymbol{x}; \boldsymbol{w}) + \epsilon, \qquad \epsilon \sim \mathcal{N}(0, \sigma^2).$$

Then the label itself is Gaussian around the network output, $p(y \,|\, \boldsymbol{x}, \boldsymbol{w}, \sigma) = \mathcal{N}(y \,|\, g(\boldsymbol{x}; \boldsymbol{w}), \sigma^2)$, and the best prediction (the conditional mean) is the network output: $\mathbb{E}[y \,|\, \boldsymbol{x}] = \int y\, p(y \,|\, \boldsymbol{x})\, \mathrm{d}y = g(\boldsymbol{x}; \boldsymbol{w})$.

If the N data points are drawn independently, their joint probability — the *likelihood* of the weights — is the product

$$p(\boldsymbol{y} \,|\, \boldsymbol{X}, \boldsymbol{w}, \sigma) = \prod_{n=1}^{N} \mathcal{N}(y^n \,|\, \boldsymbol{w}^\mathsf{T}\boldsymbol{x}^n, \sigma^2) = \prod_{n=1}^{N} \frac{1}{\sqrt{2\pi\sigma^2}} \exp\!\Big(-\frac{(y^n - \boldsymbol{w}^\mathsf{T}\boldsymbol{x}^n)^2}{2\sigma^2}\Big).$$

Taking the logarithm (which does not move the location of the maximum) turns the product into a sum:

$$\log p(\boldsymbol{y} \,|\, \boldsymbol{X}, \boldsymbol{w}, \sigma) = -\frac{N}{2}\log(2\pi\sigma^2) - \frac{1}{\sigma^2}\, S(\boldsymbol{w}), \qquad S(\boldsymbol{w}) = \frac{1}{2}\sum_{n=1}^{N} \underbrace{(y^n - \boldsymbol{w}^\mathsf{T}\boldsymbol{x}^n)}_{=:\,\epsilon^n}{}^2.$$

The first term does not depend on $\boldsymbol{w}$, and σ^2 only scales the second. So maximizing the likelihood is the same as minimizing $S(\boldsymbol{w})$, the (halved) *sum of squared residuals* ϵ^n.

Maximum likelihood ⇔ least squares

Under the assumption of additive Gaussian noise, maximizing the likelihood of the data is equivalent to minimizing the **squared loss**

$$\mathcal{L}(y, g(\boldsymbol{x}; \boldsymbol{w})) = \tfrac{1}{2}\big(y - g(\boldsymbol{x}; \boldsymbol{w})\big)^2,$$

$$R_{\text{emp}}(\boldsymbol{w}) = S(\boldsymbol{w}) = \tfrac{1}{2}\sum_{n=1}^{N}(y^n - \boldsymbol{w}^\mathsf{T}\boldsymbol{x}^n)^2 = \tfrac{1}{2}(\boldsymbol{y} - \boldsymbol{X}\boldsymbol{w})^\mathsf{T}(\boldsymbol{y} - \boldsymbol{X}\boldsymbol{w}).$$

A candidate weight vector $\tilde{\boldsymbol{w}}$ with a low sum of squared residuals $S(\tilde{\boldsymbol{w}})$ has a high likelihood.

$\mathcal{N}(0, \sigma)$ on the slides

The slides write the noise as $\epsilon \sim \mathcal{N}(0, \sigma)$, with σ the *standard deviation*. These notes write the second argument as the variance, $\mathcal{N}(0, \sigma^2)$. The formulas are the same either way — just do not plug σ in where σ^2 belongs.

4.1.2 Learning: Empirical Risk Minimization

Learning now means finding the weights that minimize the error on the training set,

$$\hat{\boldsymbol{w}} = \arg\min_{\tilde{\boldsymbol{w}}} S(\tilde{\boldsymbol{w}}),$$

an approach called **empirical risk minimization** (ERM) with $R_{\text{emp}}(\tilde{\boldsymbol{w}}, \boldsymbol{X}, \boldsymbol{y}) = S(\tilde{\boldsymbol{w}})$. Equivalently, we minimize the *negative log-likelihood* $-\log p(\boldsymbol{y} \mid \boldsymbol{X}, \boldsymbol{w}, \sigma) = \frac{N}{2}\log(2\pi\sigma^2) + \frac{1}{\sigma^2} S(\boldsymbol{w})$. Its gradient is

$$\nabla_{\boldsymbol{w}}(-\log p(\boldsymbol{y} \mid \boldsymbol{X}, \boldsymbol{w}, \sigma)) = \frac{1}{\sigma^2}\sum_{n=1}^{N}(\boldsymbol{w}^\mathsf{T}\boldsymbol{x}^n - y^n)\,\boldsymbol{x}^n = \frac{1}{\sigma^2}(\boldsymbol{X}^\mathsf{T}\boldsymbol{X}\boldsymbol{w} - \boldsymbol{X}^\mathsf{T}\boldsymbol{y}).$$

Note the structure of each summand: *(prediction − target) × input*. We will meet it again for every single-layer network.

There are two ways to find the minimum: solve for it directly, or walk towards it step by step.

4.1.3 Closed-Form Solution: The Normal Equations

Setting the gradient to zero gives the *normal equations*:

$$0 = \boldsymbol{X}^\mathsf{T}\boldsymbol{X}\boldsymbol{w} - \boldsymbol{X}^\mathsf{T}\boldsymbol{y} \quad\Longleftrightarrow\quad \boldsymbol{X}^\mathsf{T}\boldsymbol{X}\,\boldsymbol{w} = \boldsymbol{X}^\mathsf{T}\boldsymbol{y} \quad\Longrightarrow\quad \hat{\boldsymbol{w}} = (\boldsymbol{X}^\mathsf{T}\boldsymbol{X})^{-1}\boldsymbol{X}^\mathsf{T}\boldsymbol{y}.$$

Least-squares solution

If $\boldsymbol{X}^\mathsf{T}\boldsymbol{X}$ has full rank (is invertible), the unique minimizer of the squared error — and the maximum-likelihood estimator under Gaussian noise — is

$$\hat{\boldsymbol{w}} = (\boldsymbol{X}^\mathsf{T}\boldsymbol{X})^{-1}\boldsymbol{X}^\mathsf{T}\boldsymbol{y} = \boldsymbol{X}^{+}\boldsymbol{y}, \qquad \boldsymbol{X}^{+} = (\boldsymbol{X}^\mathsf{T}\boldsymbol{X})^{-1}\boldsymbol{X}^\mathsf{T},$$

where $\boldsymbol{X}^{+}$ is the **Moore–Penrose pseudo-inverse** of $\boldsymbol{X}$. For centered data, $\frac{1}{N}\boldsymbol{X}^\mathsf{T}\boldsymbol{X}$ is the estimate of the covariance matrix of the features.

Fitting a line through three points

Data: $x = 0, 1, 2$ with labels $\boldsymbol{y} = (1, 3, 4)^\mathsf{T}$. To fit an intercept, we append the constant feature, so each row of $\boldsymbol{X}$ is $(1, x^n)$:

$$\boldsymbol{X} = \begin{pmatrix} 1 & 0 \\ 1 & 1 \\ 1 & 2 \end{pmatrix}, \quad \boldsymbol{X}^\mathsf{T}\boldsymbol{X} = \begin{pmatrix} 3 & 3 \\ 3 & 5 \end{pmatrix}, \quad \boldsymbol{X}^\mathsf{T}\boldsymbol{y} = \begin{pmatrix} 8 \\ 11 \end{pmatrix}, \quad (\boldsymbol{X}^\mathsf{T}\boldsymbol{X})^{-1} = \frac{1}{6}\begin{pmatrix} 5 & -3 \\ -3 & 3 \end{pmatrix}.$$

Hence $\hat{\boldsymbol{w}} = \frac{1}{6}(40 - 33,\ -24 + 33)^\mathsf{T} = (7/6,\ 3/2)^\mathsf{T} \approx (1.167, 1.5)^\mathsf{T}$, i.e. the line $\hat{y} = 1.167 + 1.5\,x$. The predictions are $(1.167, 2.667, 4.167)$, the residuals $(-0.167, 0.333, -0.167)$, and $S(\hat{\boldsymbol{w}}) = \frac{1}{2}(0.0278 + 0.1111 + 0.0278) = 0.083$. No other line through these points has a smaller squared error.

4.1.4 Gradient Descent and Stochastic Gradient Descent

Instead of solving the normal equations, we can start from an initial guess and improve it repeatedly by stepping against the gradient:

$$\boldsymbol{w}^{\text{new}} = \boldsymbol{w}^{\text{old}} - \eta \, \nabla_{\boldsymbol{w}} S(\boldsymbol{w})\big|_{\boldsymbol{w}^{\text{old}}}, \qquad \nabla_{\boldsymbol{w}} S(\boldsymbol{w}) = \sum_{n=1}^{N} (\boldsymbol{w}^{\mathsf{T}} \boldsymbol{x}^n - y^n)\, \boldsymbol{x}^n.$$

This is **gradient descent**. The *learning rate* η is critical: too small and the algorithm crawls, too large and it overshoots and diverges. For least squares, gradient descent works well because the problem is *convex*: there are no bad local minima to get stuck in.

One gradient step on the three points

Start at $\boldsymbol{w} = (0, 0)^{\mathsf{T}}$, where $S = \frac{1}{2}(1 + 9 + 16) = 13$. The gradient is $\boldsymbol{X}^{\mathsf{T}}(\boldsymbol{X}\boldsymbol{w} - \boldsymbol{y}) = -\boldsymbol{X}^{\mathsf{T}}\boldsymbol{y} = (-8, -11)^{\mathsf{T}}$. With $\eta = 0.1$ the new weights are $\boldsymbol{w} = (0.8, 1.1)^{\mathsf{T}}$, the predictions $(0.8, 1.9, 3.0)$, and the error drops to $S = \frac{1}{2}(0.04 + 1.21 + 1.00) = 1.125$. Repeating the step converges to the closed-form solution $(1.167, 1.5)^{\mathsf{T}}$ with $S = 0.083$.

For large data sets, computing the full gradient over all N samples for every single step is expensive. Here a simple observation helps: the scaled gradient is a *mean* over data points,

$$\frac{1}{N} \nabla_{\boldsymbol{w}} S(\boldsymbol{w}) = \frac{1}{N} \sum_{n=1}^{N} (\boldsymbol{w}^{\mathsf{T}} \boldsymbol{x}^n - y^n)\, \boldsymbol{x}^n,$$

and a mean can be estimated from a random subset. Drawing a **mini-batch**

$$B = \{\boldsymbol{x}^{n_1}, \ldots, \boldsymbol{x}^{n_{|B|}}\} \subset \{\boldsymbol{x}^1, \ldots, \boldsymbol{x}^N\}$$

gives

$$\frac{1}{N} \nabla_{\boldsymbol{w}} S(\boldsymbol{w}) \approx \frac{1}{|B|} \sum_{b=1}^{|B|} (\boldsymbol{w}^{\mathsf{T}} \boldsymbol{x}^{n_b} - y^{n_b})\, \boldsymbol{x}^{n_b},$$

and using this noisy but cheap estimate in the update is **stochastic gradient descent** (SGD). It is the workhorse of all deep learning (Section 4.10).

4.1.5 The Trained Network

The goal of learning is a trained network that makes predictions for new inputs, $g(\boldsymbol{x}^{\text{new}}; \boldsymbol{w}) = \boldsymbol{w}^{\mathsf{T}} \boldsymbol{x}^{\text{new}}$. In the slides' example, the weights $\boldsymbol{w} = (0.1, -1.0, 0.4)^{\mathsf{T}}$ and the new input $\boldsymbol{x}^{\text{new}} = (1.0, -0.3, 0)^{\mathsf{T}}$ give the prediction

$$a = 0.1 \cdot 1.0 + (-1.0) \cdot (-0.3) + 0.4 \cdot 0 = 0.4.$$

Intuition & Mental Model

The linear neuron already contains the whole pipeline of this course in miniature: an *architecture* (weighted sum), an *objective* (empirical error, motivated by maximum likelihood), *learning* (minimizing the objective — in closed form or by gradient descent), and the *goal*: a trained network that predicts well on new data. Every network that follows changes one of these pieces.

4.2 Perceptron

Now the labels are not real numbers but two classes, coded as $y \in \{-1, +1\}$: diseased vs. healthy, spam vs. normal email, win or loss. We could simply run linear regression on the labels ± 1, but this is a poor fit to the problem: the squared loss also punishes points that are classified correctly with a large margin (a prediction of $+5$ for a label $+1$ is "wrong" by 4), and the output is a real number, not a class.

Perceptron

The perceptron is a linear neuron followed by the sign function:

$$g(\boldsymbol{x}; \boldsymbol{w}) = a = \operatorname{sign}(\boldsymbol{w}^\mathsf{T}\boldsymbol{x}), \qquad \operatorname{sign}(z) = \begin{cases} +1 & z > 0, \\ 0 & z = 0, \\ -1 & z < 0. \end{cases}$$

It predicts class $+1$ on one side of the hyperplane $\boldsymbol{w}^\mathsf{T}\boldsymbol{x} = 0$ and class -1 on the other. The weight vector $\boldsymbol{w}$ is the *normal vector* of this decision boundary and points to the $+1$ side.

4.2.1 Perceptron Loss and Learning Rule

A sample is classified correctly if y and $\boldsymbol{w}^\mathsf{T}\boldsymbol{x}$ have the same sign, i.e. if $y\,\boldsymbol{w}^\mathsf{T}\boldsymbol{x} > 0$. The **perceptron loss** is zero for correct samples and grows linearly with how far a misclassified sample is on the wrong side:

$$\mathcal{L}(y, \boldsymbol{w}^\mathsf{T}\boldsymbol{x}) = \max(0,\ -y\,\boldsymbol{w}^\mathsf{T}\boldsymbol{x}), \qquad R_{\text{emp}}(\boldsymbol{y}, \boldsymbol{X}, \boldsymbol{w}) = -\sum_{n \in \mathcal{M}} y^n\, \boldsymbol{w}^\mathsf{T}\boldsymbol{x}^n,$$

where $\mathcal{M}$ is the set of **misclassified** samples (the slides write the sum as running over all n with $a^n y^n = -1$). Each term is positive, so $R_{\text{emp}} \geq 0$, with equality exactly when all samples are classified correctly.

The gradient is $\nabla_{\boldsymbol{w}} R_{\text{emp}} = -\sum_{n \in \mathcal{M}} y^n \boldsymbol{x}^n$, and a gradient-descent step gives the **perceptron learning rule**:

$$\Delta\boldsymbol{w} = \eta \sum_{n \in \mathcal{M}} y^n\, \boldsymbol{x}^n \quad \text{(whole data set)}, \qquad \Delta\boldsymbol{w} = \eta\, y\, \boldsymbol{x} \ \text{ if } \boldsymbol{x} \text{ is misclassified} \quad \text{(single sample)}.$$

In the online version, we go through the samples one by one and change the weights only when the current sample is misclassified.

Sign conventions on the slides

The slides write the perceptron loss as "$y(\boldsymbol{w}^\mathsf{T}\boldsymbol{x})$ for misclassified samples"; with that sign it would be *negative*. The loss must be $-y\,\boldsymbol{w}^\mathsf{T}\boldsymbol{x}$ (as in the slides' empirical error, which has the minus sign). Also, with $\operatorname{sign}(0) = 0$, a sample exactly on the boundary would count as correct, and a perceptron started at $\boldsymbol{w} = \boldsymbol{0}$ would never move. In practice — and in the convergence proof — a sample counts as misclassified if $y\,\boldsymbol{w}^\mathsf{T}\boldsymbol{x} \leq 0$.

4.2.2 Geometric Interpretation

The learning rule has a nice geometric meaning (Fig. 4.2). If a sample $\boldsymbol{x}$ of class $+1$ lies on the wrong side, then $\boldsymbol{w}^\mathsf{T}\boldsymbol{x} < 0$: the angle between $\boldsymbol{w}$ and $\boldsymbol{x}$ is larger than $90°$. Adding $\eta\boldsymbol{x}$ rotates $\boldsymbol{w}$ *towards* $\boldsymbol{x}$, and the boundary rotates with it. For a misclassified sample of class -1, the update $\boldsymbol{w} - \eta\boldsymbol{x}$ rotates $\boldsymbol{w}$ *away* from $\boldsymbol{x}$. Either way, $y\,\boldsymbol{w}^\mathsf{T}\boldsymbol{x}$ increases by $\eta\,\|\boldsymbol{x}\|^2$: the update makes this particular sample "more correct". It may, however, make other samples wrong.

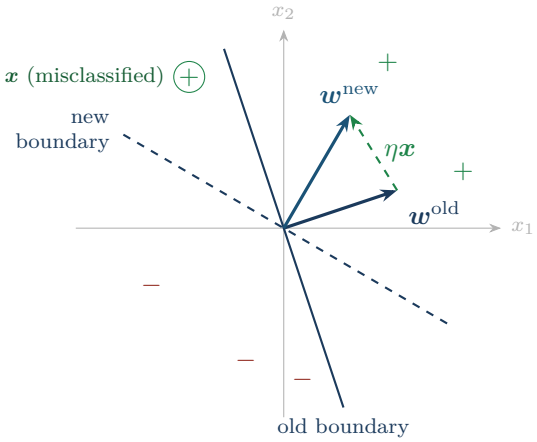


Figure 4.2: One perceptron update. The + sample $\boldsymbol{x}$ lies on the negative side of the old boundary (solid). Adding $\eta\boldsymbol{x}$ to the weight vector rotates $\boldsymbol{w}$ towards $\boldsymbol{x}$; the new boundary (dashed) is perpendicular to $\boldsymbol{w}^{\text{new}}$ and now classifies all points correctly. Here $\boldsymbol{w}^{\text{old}} = (1.2, 0.4)$, $\boldsymbol{x} = (-1, 1.6)$, $\eta = 0.5$.

Running the perceptron by hand

Four samples in 2D, each extended by a constant feature 1 for the bias: $\boldsymbol{x}^1 = (2, 1, 1)$, $\boldsymbol{x}^2 = (0, 2, 1)$ with $y = +1$ and $\boldsymbol{x}^3 = (-1, 0, 1)$, $\boldsymbol{x}^4 = (1, -2, 1)$ with $y = -1$. Start with $\boldsymbol{w} = (0, 0, 0)$, $\eta = 1$, and update whenever $y\,\boldsymbol{w}^\mathsf{T}\boldsymbol{x} \le 0$.

epoch	sample	$\boldsymbol{w}$ before	$y\,\boldsymbol{w}^\mathsf{T}\boldsymbol{x}$	action
1	$\boldsymbol{x}^1$	$(0, 0, 0)$	0	update: $\boldsymbol{w} = (2, 1, 1)$
1	$\boldsymbol{x}^2$	$(2, 1, 1)$	3	correct
1	$\boldsymbol{x}^3$	$(2, 1, 1)$	1	correct
1	$\boldsymbol{x}^4$	$(2, 1, 1)$	-1	update: $\boldsymbol{w} = (2, 1, 1) - (1, -2, 1) = (1, 3, 0)$
2	all	$(1, 3, 0)$	$5, 6, 1, 5$	all correct — stop

After two updates the perceptron has found the separating line $x_1 + 3x_2 = 0$. A different order of the samples would have found a different line.

4.2.3 Convergence for Linearly Separable Data

Does the learning rule always find a solution? If one exists, yes.

Perceptron convergence theorem

If the training data are **linearly separable** — there is a $\hat{\boldsymbol{w}}$ with $y^n\,\hat{\boldsymbol{w}}^\mathsf{T}\boldsymbol{x}^n > 0$ for all n — then the perceptron learning rule finds a separating weight vector after a *finite* number of updates. With margin $\gamma = \min_n y^n\,\hat{\boldsymbol{w}}^\mathsf{T}\boldsymbol{x}^n$ and $R = \max_n \|\boldsymbol{x}^n\|$, the number of updates is at most $\|\hat{\boldsymbol{w}}\|^2\,R^2/\gamma^2$.

Proof idea. Start with $\boldsymbol{w} = \boldsymbol{0}$ and update with $\eta = 1$ (the step size does not matter when starting from zero). If sample n has been misclassified τ_n times, then after $\tau = \sum_n \tau_n$ updates $\boldsymbol{w} = \sum_n \tau_n\,y^n\boldsymbol{x}^n$. Now compare two quantities:

1. *The projection onto $\hat{\boldsymbol{w}}$ grows linearly.* Every update adds at least γ, so
$$\hat{\boldsymbol{w}}^\mathsf{T}\boldsymbol{w} = \sum_n \tau_n\,\hat{\boldsymbol{w}}^\mathsf{T}\boldsymbol{x}^n y^n \geq \tau\gamma.$$
2. *The length grows only like $\sqrt{\tau}$.* An update on a misclassified sample changes the squared length by $\|\boldsymbol{w} + y^n\boldsymbol{x}^n\|^2 - \|\boldsymbol{w}\|^2 = \|\boldsymbol{x}^n\|^2 + 2y^n\boldsymbol{w}^\mathsf{T}\boldsymbol{x}^n \leq R^2$, because the last term is ≤ 0 for a misclassified sample. So $\|\boldsymbol{w}\|^2 \leq \tau R^2$.

By Cauchy–Schwarz, $\tau\gamma \leq \hat{\boldsymbol{w}}^\mathsf{T}\boldsymbol{w} \leq \|\hat{\boldsymbol{w}}\|\,\|\boldsymbol{w}\| \leq \|\hat{\boldsymbol{w}}\|\,\sqrt{\tau}R$. The left side grows like τ, the right side only like $\sqrt{\tau}$ — for large τ the two would be incompatible. Hence $\tau \leq \|\hat{\boldsymbol{w}}\|^2\,R^2/\gamma^2$. □

What the theorem does *not* say

- **The solution is not unique.** Any separating hyperplane has zero perceptron loss. Which one the algorithm finds depends on the initialization and on the order of the samples.
- **Non-separable data:** if no separating hyperplane exists, the perceptron never stops updating — the weights keep cycling, and the algorithm does not converge.
- The bound can be large: a small margin γ means many updates.

4.3 Linear Neuron with Sigmoid Activation: Logistic Regression

Again we have two classes, now coded as $y \in \{0, 1\}$. Instead of a hard decision we would like the network to output a *probability* for class 1. The trick: squash the weighted sum into $(0, 1)$.

Logistic neuron (logistic regression)

$$g(\boldsymbol{x}; \boldsymbol{w}) = a = \sigma(\boldsymbol{w}^\mathsf{T}\boldsymbol{x}) = \frac{1}{1 + e^{-\boldsymbol{w}^\mathsf{T}\boldsymbol{x}}},$$

where σ is the **logistic sigmoid**. The output is interpreted as $p(y = 1 \,|\, \boldsymbol{x}; \boldsymbol{w})$. This is the model of logistic regression.

The sigmoid (Fig. 4.3) is 0.5 at zero, approaches 1 for large positive and 0 for large negative inputs, and is symmetric: $\sigma(-z) = 1 - \sigma(z)$. Its derivative has a particularly convenient form, which we will use over and over:

$$\sigma'(z) = \frac{\mathrm{d}}{\mathrm{d}z}\frac{1}{1 + e^{-z}} = \frac{e^{-z}}{(1 + e^{-z})^2} = \sigma(z)(1 - \sigma(z)).$$

It is largest at $z = 0$, where $\sigma'(0) = 0.5 \cdot 0.5 = 0.25$, and goes to zero for large $|z|$ — the sigmoid *saturates*.

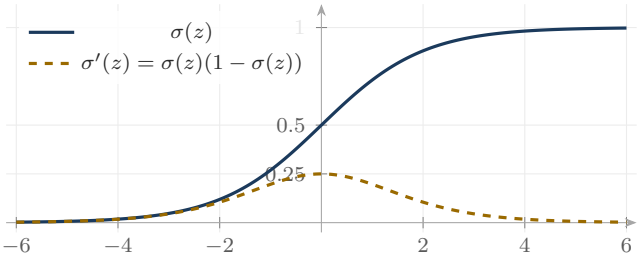


Figure 4.3: The logistic sigmoid and its derivative. The derivative never exceeds 0.25 — a fact that becomes important for the vanishing gradient problem (Section 4.7.3).

4.3.1 Likelihood and Binary Cross-Entropy

The recipe is the same as for the linear neuron: write down the probability of the data and maximize it. The network says $p(y = 1 \,|\, \boldsymbol{x}; \boldsymbol{w}) = g(\boldsymbol{x}; \boldsymbol{w})$ and $p(y = 0 \,|\, \boldsymbol{x}; \boldsymbol{w}) = 1 - g(\boldsymbol{x}; \boldsymbol{w})$, which can be written compactly as a Bernoulli distribution

$$p(y \,|\, \boldsymbol{x}; \boldsymbol{w}) = g(\boldsymbol{x}; \boldsymbol{w})^{y} \left(1 - g(\boldsymbol{x}; \boldsymbol{w})\right)^{1-y}.$$

(For $y = 1$ the second factor is 1, for $y = 0$ the first.) For independent samples, the likelihood and the negative log-likelihood are

$$p(\boldsymbol{y} \,|\, \boldsymbol{X}, \boldsymbol{w}) = \prod_{n=1}^{N} g(\boldsymbol{x}^n; \boldsymbol{w})^{y^n} \left(1 - g(\boldsymbol{x}^n; \boldsymbol{w})\right)^{1-y^n},$$

$$-\log p(\boldsymbol{y} \,|\, \boldsymbol{X}, \boldsymbol{w}) = -\sum_{n=1}^{N} \Big(y^n \log g(\boldsymbol{x}^n; \boldsymbol{w}) + (1 - y^n) \log\big(1 - g(\boldsymbol{x}^n; \boldsymbol{w})\big) \Big).$$

Binary cross-entropy

The loss

$$\mathcal{L}(y, g(\boldsymbol{x}; \boldsymbol{w})) = -\Big(y \log g(\boldsymbol{x}; \boldsymbol{w}) + (1 - y) \log\big(1 - g(\boldsymbol{x}; \boldsymbol{w})\big) \Big)$$

is called **binary cross-entropy**. It is the negative log-probability that the model assigns to the correct class: small if the model is confidently right, very large if it is confidently wrong.

4.3.2 Gradient and Hessian

Using $\sigma' = \sigma(1 - \sigma)$ and writing $a^n = \sigma(\boldsymbol{w}^\mathsf{T} \boldsymbol{x}^n)$:

$$\begin{aligned} \nabla_{\boldsymbol{w}} R_{\text{emp}} &= -\sum_{n=1}^{N} \Big(\frac{y^n}{a^n} - \frac{1 - y^n}{1 - a^n} \Big) \nabla_{\boldsymbol{w}} \sigma(\boldsymbol{w}^\mathsf{T} \boldsymbol{x}^n) = -\sum_{n=1}^{N} \Big(\frac{y^n}{a^n} - \frac{1 - y^n}{1 - a^n} \Big) a^n (1 - a^n)\, \boldsymbol{x}^n \\ &= -\sum_{n=1}^{N} \big(y^n (1 - a^n) - (1 - y^n) a^n \big) \boldsymbol{x}^n = \sum_{n=1}^{N} \big(\sigma(\boldsymbol{w}^\mathsf{T} \boldsymbol{x}^n) - y^n \big)\, \boldsymbol{x}^n. \end{aligned}$$

Logistic regression: gradient and Hessian

For a single sample and for the whole data set:

$$\nabla_{\boldsymbol{w}} R_{\text{emp}} = (\sigma(\boldsymbol{w}^\mathsf{T}\boldsymbol{x}) - y)\,\boldsymbol{x}, \qquad \nabla_{\boldsymbol{w}} R_{\text{emp}} = \sum_{n=1}^{N} (\sigma(\boldsymbol{w}^\mathsf{T}\boldsymbol{x}^n) - y^n)\,\boldsymbol{x}^n = \boldsymbol{X}^\mathsf{T}(\boldsymbol{a} - \boldsymbol{y}),$$

$$\nabla^2_{\boldsymbol{w}} R_{\text{emp}} = \sigma(\boldsymbol{w}^\mathsf{T}\boldsymbol{x})(1 - \sigma(\boldsymbol{w}^\mathsf{T}\boldsymbol{x}))\,\boldsymbol{x}\boldsymbol{x}^\mathsf{T}, \qquad \nabla^2_{\boldsymbol{w}} R_{\text{emp}} = \sum_{n=1}^{N} a^n(1 - a^n)\,\boldsymbol{x}^n\boldsymbol{x}^{n\mathsf{T}} = \boldsymbol{X}^\mathsf{T}\boldsymbol{R}\boldsymbol{X},$$

where $\boldsymbol{a} = (a^1, \ldots, a^N)^\mathsf{T}$ and $\boldsymbol{R} = \operatorname{diag}(a^n(1 - a^n))$. Since all diagonal entries of $\boldsymbol{R}$ are positive, $\boldsymbol{X}^\mathsf{T}\boldsymbol{R}\boldsymbol{X}$ is positive semi-definite: the binary cross-entropy is **convex** in $\boldsymbol{w}$.

The gradient looks exactly like the one of linear regression — *(prediction − target) × input* — only the prediction is now $\sigma(\boldsymbol{w}^\mathsf{T}\boldsymbol{x})$ instead of $\boldsymbol{w}^\mathsf{T}\boldsymbol{x}$. This is no coincidence, and it will save us a lot of work in backpropagation (Table 4.3).

The sign of the Hessian

One slide writes the Hessian of logistic regression as $-\boldsymbol{X}^\mathsf{T}\boldsymbol{R}\boldsymbol{X}$; that is the Hessian of the *log-likelihood*, which is maximized. For the error R_{emp} (negative log-likelihood), which is minimized, the Hessian is $+\boldsymbol{X}^\mathsf{T}\boldsymbol{R}\boldsymbol{X}$, and this is the matrix that appears in the update rule below.

4.3.3 Learning

Setting the gradient to zero gives no closed-form solution this time (the σ is in the way), so we have to iterate:

- **gradient descent:** $\boldsymbol{w}^{\text{new}} = \boldsymbol{w}^{\text{old}} - \eta\,\nabla_{\boldsymbol{w}} R_{\text{emp}}(\boldsymbol{y}, \boldsymbol{X}, \boldsymbol{w})|_{\boldsymbol{w}^{\text{old}}}$;
- **stochastic gradient descent:** the same with a mini-batch estimate $\tilde{\nabla}_{\boldsymbol{w}} R_{\text{emp}}$ of the gradient;
- **iteratively reweighted least squares** (IRLS), the method typically used in statistics software: Newton's method with the Hessian, $\boldsymbol{w}^{\text{new}} = \boldsymbol{w}^{\text{old}} - \boldsymbol{H}^{-1}\nabla_{\boldsymbol{w}} R_{\text{emp}}|_{\boldsymbol{w}^{\text{old}}}$ (derived below).

One gradient step for logistic regression

Weights $\boldsymbol{w} = (0.5, -1)^\mathsf{T}$, one sample $\boldsymbol{x} = (1, 2)^\mathsf{T}$ with label $y = 1$.

- Forward: $\boldsymbol{w}^\mathsf{T}\boldsymbol{x} = 0.5 - 2 = -1.5$, $a = \sigma(-1.5) = 0.182$. The model gives the correct class only 18 % probability; the loss is $-\log 0.182 = 1.70$.
- Gradient: $(a - y)\boldsymbol{x} = (0.182 - 1)(1, 2)^\mathsf{T} = (-0.818, -1.635)^\mathsf{T}$.
- Step with $\eta = 0.5$: $\boldsymbol{w} = (0.5, -1)^\mathsf{T} + 0.5\,(0.818, 1.635)^\mathsf{T} = (0.909, -0.182)^\mathsf{T}$.
- Check: $\boldsymbol{w}^\mathsf{T}\boldsymbol{x} = 0.544$, $a = \sigma(0.544) = 0.633$, loss 0.458.

One step moved the prediction from "probably class 0" to "probably class 1". Note how large the step was: the gradient is proportional to the error $a - y$, which was big.

4.3.4 Intermezzo: Optimization and Newton's Method

Learning is an optimization problem, so it is worth placing it in the general picture. A generic (constrained) optimization problem reads

$$\min_{\boldsymbol{x}} \; f(\boldsymbol{x}) \quad \text{s.t.} \quad g(\boldsymbol{x}) \le 0, \;\; h(\boldsymbol{x}) = 0, \qquad f : \mathbb{R}^n \to \mathbb{R}, \; g : \mathbb{R}^n \to \mathbb{R}^m, \; h : \mathbb{R}^n \to \mathbb{R}.$$

(This is general optimization notation, not the course's ML notation.) In neural-network training, the variable $\boldsymbol{x}$ is our *weight vector*, f is the *empirical error*, and there are typically *no constraints*. Optimization methods differ in what they may ask about f:

- **black-box optimization:** only function values $f(\boldsymbol{x})$;
- **gradient-based optimization:** $f(\boldsymbol{x})$ and the gradient $\nabla f(\boldsymbol{x})$ — this is what deep learning uses;
- **second-order optimization:** additionally the Hessian $\nabla^2 f(\boldsymbol{x})$.

Newton's method. To find a *root* of a function, approximate it by its tangent (a first-order Taylor expansion) and solve for the zero of the tangent:

$$f(x) \approx f(x_0) + (x - x_0) f'(x_0) = 0 \quad \Longrightarrow \quad x = x_0 - \frac{f(x_0)}{f'(x_0)}.$$

To find an *optimum*, apply the same idea to the derivative (an optimum is a root of f'). In one and in n dimensions:

$$x^{\text{new}} = x^{\text{old}} - \frac{f'(x^{\text{old}})}{f''(x^{\text{old}})}, \qquad \boldsymbol{x}^{\text{new}} = \boldsymbol{x}^{\text{old}} - \left(\nabla^2 f(\boldsymbol{x}^{\text{old}})\right)^{-1} \nabla f(\boldsymbol{x}^{\text{old}}).$$

Compared with gradient descent, the Hessian replaces the fixed learning rate by a step that is adapted to the curvature in every direction.

Newton's method for linear and logistic regression

With $\boldsymbol{w}^{\text{new}} = \boldsymbol{w}^{\text{old}} - (\nabla^2_{\boldsymbol{w}} R_{\text{emp}})^{-1} \nabla_{\boldsymbol{w}} R_{\text{emp}}$:

- **Linear regression:** $\nabla_{\boldsymbol{w}} R_{\text{emp}} = \boldsymbol{X}^\top \boldsymbol{X} \boldsymbol{w} - \boldsymbol{X}^\top \boldsymbol{y}$, $\nabla^2_{\boldsymbol{w}} R_{\text{emp}} = \boldsymbol{X}^\top \boldsymbol{X}$, so

  $$\boldsymbol{w}^{\text{new}} = \boldsymbol{w}^{\text{old}} - (\boldsymbol{X}^\top \boldsymbol{X})^{-1} (\boldsymbol{X}^\top \boldsymbol{X} \boldsymbol{w}^{\text{old}} - \boldsymbol{X}^\top \boldsymbol{y}) = (\boldsymbol{X}^\top \boldsymbol{X})^{-1} \boldsymbol{X}^\top \boldsymbol{y}.$$

  A single Newton step from *any* starting point lands exactly on the least-squares solution — the error is quadratic, so its second-order Taylor expansion is exact.
- **Logistic regression:** $\nabla_{\boldsymbol{w}} R_{\text{emp}} = \boldsymbol{X}^\top (\boldsymbol{a} - \boldsymbol{y})$, $\nabla^2_{\boldsymbol{w}} R_{\text{emp}} = \boldsymbol{X}^\top \boldsymbol{R} \boldsymbol{X}$, so

  $$\boldsymbol{w}^{\text{new}} = \boldsymbol{w}^{\text{old}} - (\boldsymbol{X}^\top \boldsymbol{R} \boldsymbol{X})^{-1} \boldsymbol{X}^\top (\boldsymbol{a} - \boldsymbol{y}).$$

  This is **iteratively reweighted least squares**: every step solves a weighted least-squares problem, and the weights $\boldsymbol{R}$ change in each iteration because they depend on the current predictions.

Logistic regression on separable data

If the two classes are *linearly separable*, the binary cross-entropy can be pushed arbitrarily close to zero by scaling up any separating $\boldsymbol{w}$: the sigmoid becomes steeper and steeper, and the maximum-likelihood solution does not exist ($\|\boldsymbol{w}\| \to \infty$). On non-separable data

(the typical case, shown on the slides) a finite optimum exists. Regularization (Chapter 8) is the standard remedy for the separable case.

A real-world example. In 2019, the Austrian public employment service (AMS) announced that an algorithm would be used to assess job-seekers. The model was a logistic regression on several person-centric variables (gender, age, occupational group, education, citizenship group, impairment, employment history, and their interactions), estimating a job-seeker's short-term chance of re-employment (90 days of employment within seven months). It was shown to be unfair, for example with respect to women. The example is a reminder that even the simplest models in this chapter are used for high-stakes decisions — and that a model fitted by maximum likelihood faithfully reproduces the patterns, including the biases, of its training data.

4.4 Linear Neuron with Softmax Activation: Softmax Regression

Next, the labels can take one of K classes: cars, dogs, cats, airplanes, ... in image recognition. Coding the classes as numbers $1, \dots, K$ and running regression would impose a meaningless order (is a cat "between" a dog and an airplane?). Instead, each label becomes a vector.

One-hot encoding

A label of class k out of K classes is coded as the vector $\boldsymbol{y} \in \{0,1\}^K$ with $y_k = 1$ and all other entries 0. For example, with classes A, B, C, D: A $\mapsto (1,0,0,0)$, C $\mapsto (0,0,1,0)$. The same encoding is used for categorical *input* features (Section 4.10).

Softmax regression

The network has K output units and a *weight matrix* $\boldsymbol{W} \in \mathbb{R}^{K \times D}$ instead of a weight vector:

$$g(\boldsymbol{x}; \boldsymbol{W}) = \boldsymbol{a} = \operatorname{softmax}(\boldsymbol{W}\boldsymbol{x}), \qquad \operatorname{softmax}(\boldsymbol{s}) = \Big(\frac{e^{s_1}}{\sum_{j=1}^{K} e^{s_j}}, \dots, \frac{e^{s_K}}{\sum_{j=1}^{K} e^{s_j}} \Big)^{\mathsf{T}}.$$

The output is a **probability vector**: all entries are positive and sum to one, and a_k is the predicted probability of class k. The weight w_{kd} connects input d to output k; row k of $\boldsymbol{W}$ is the weight vector $\boldsymbol{w}_k$ of output unit k.

4.4.1 Categorical Cross-Entropy

Same recipe once more. The probability the model assigns to a one-hot label is $p(\boldsymbol{y} \mid \boldsymbol{a}) = \prod_{k=1}^{K} a_k^{y_k}$ (only the factor of the correct class is not 1), and its negative logarithm is the loss.

Categorical cross-entropy

$$\mathcal{L}(\boldsymbol{y}, \boldsymbol{a}) = -\log p(\boldsymbol{y} \mid \boldsymbol{a}) = -\sum_{k=1}^{K} y_k \log a_k, \qquad R_{\text{emp}}(\boldsymbol{W}, \boldsymbol{X}, \boldsymbol{Y}) = -\sum_{n=1}^{N} \sum_{k=1}^{K} y_k^n \log a_k^n.$$

For a one-hot label, only one term survives: the loss is minus the log of the predicted probability of the correct class.

4.4.2 Gradient

The derivative of the softmax is *coupled*: each output depends on all pre-activations $s_j = \boldsymbol{w}_j^\mathsf{T}\boldsymbol{x}$,

$$\frac{\partial a_k}{\partial s_j} = a_k(\delta_{kj} - a_j) \qquad (\delta_{kj} = 1 \text{ if } k = j, \text{ else } 0).$$

Combining with the cross-entropy and using $\sum_k y_k = 1$:

$$\frac{\partial \mathcal{L}}{\partial s_j} = -\sum_{k=1}^{K} y_k \frac{1}{a_k} a_k(\delta_{kj} - a_j) = -y_j + a_j \sum_k y_k = a_j - y_j.$$

Since $\partial s_\xi / \partial w_{\xi d} = x_d$, the derivative with respect to a single weight and the gradient with respect to the whole matrix are

$$\frac{\partial \mathcal{L}}{\partial w_{\xi d}} = (a_\xi - y_\xi)\, x_d, \qquad \nabla_{\boldsymbol{W}} \mathcal{L} = (\boldsymbol{a} - \boldsymbol{y})\, \boldsymbol{x}^\mathsf{T}.$$

(The full derivation is part of the exercises.)

Softmax and cross-entropy with numbers

Three classes, pre-activations $\boldsymbol{s} = \boldsymbol{W}\boldsymbol{x} = (2, 1, 0.1)^\mathsf{T}$, and the true class is the second one, $\boldsymbol{y} = (0, 1, 0)^\mathsf{T}$.

- $e^{\boldsymbol{s}} = (7.389, 2.718, 1.105)$, sum 11.212, so $\boldsymbol{a} = \text{softmax}(\boldsymbol{s}) = (0.659, 0.242, 0.099)^\mathsf{T}$ (sums to 1).
- Loss: $-\log a_2 = -\log 0.242 = 1.417$.
- Error signal: $\boldsymbol{a} - \boldsymbol{y} = (0.659, -0.758, 0.099)^\mathsf{T}$. The gradient $(\boldsymbol{a} - \boldsymbol{y})\boldsymbol{x}^\mathsf{T}$ pushes the weights of class 2 *up* (towards $\boldsymbol{x}$) and those of classes 1 and 3 *down*, the more so the larger their wrong probability.

Type of optimization problem and convergence. As for logistic regression, the categorical cross-entropy is convex in $\boldsymbol{W}$, there is no closed-form solution, and gradient descent (or Newton's method) is used. Two remarks answer the slides' questions about uniqueness and convergence:

- The softmax does not change if the same number is added to all pre-activations, $\text{softmax}(\boldsymbol{s} + c\mathbf{1}) = \text{softmax}(\boldsymbol{s})$. Adding the same vector $\boldsymbol{v}$ to every row of $\boldsymbol{W}$ adds $\boldsymbol{v}^\mathsf{T}\boldsymbol{x}$ to every s_k, so infinitely many weight matrices give the same model: the minimizer is never unique (it is unique only up to such shifts).
- As in the binary case, perfectly separable data let the weights grow without bound.

Intuition & Mental Model

For $K = 2$ the softmax reduces to the sigmoid: $a_1 = e^{s_1}/(e^{s_1} + e^{s_2}) = 1/(1 + e^{-(s_1 - s_2)}) = \sigma(s_1 - s_2)$. Softmax regression is logistic regression with K classes; only the *difference* of the pre-activations matters — which is exactly the shift invariance above.

4.4.3 Comparing the Single-Layer Networks

Put the three gradients next to each other:

Same gradient, three models

$$\begin{aligned} \text{linear regression:} \quad & \nabla_{\boldsymbol{w}} R_{\text{emp}}(y, \boldsymbol{x}, \boldsymbol{w}) = (\boldsymbol{w}^\mathsf{T}\boldsymbol{x} - y)\,\boldsymbol{x}, \\ \text{logistic regression:} \quad & \nabla_{\boldsymbol{w}} R_{\text{emp}}(y, \boldsymbol{x}, \boldsymbol{w}) = (\sigma(\boldsymbol{w}^\mathsf{T}\boldsymbol{x}) - y)\,\boldsymbol{x}, \\ \text{softmax regression:} \quad & \nabla_{\boldsymbol{W}} R_{\text{emp}}(\boldsymbol{y}, \boldsymbol{x}, \boldsymbol{W}) = (\text{softmax}(\boldsymbol{W}\boldsymbol{x}) - \boldsymbol{y})\,\boldsymbol{x}^\mathsf{T}. \end{aligned}$$

In all three cases: **(prediction − target) × input**. The pairs (linear output, squared loss), (sigmoid output, binary cross-entropy) and (softmax output, categorical cross-entropy) are "matched" — in statistics they are called *canonical links* — and the derivative of the activation cancels against the derivative of the loss.

Network	Labels	Output	Loss	Learning
linear neuron	$y \in \mathbb{R}$	$\boldsymbol{w}^\mathsf{T}\boldsymbol{x}$	squared loss	closed form, GD, SGD
perceptron	$y \in \{-1, 1\}$	$\text{sign}(\boldsymbol{w}^\mathsf{T}\boldsymbol{x})$	perceptron loss	perceptron rule
logistic regression	$y \in \{0, 1\}$	$\sigma(\boldsymbol{w}^\mathsf{T}\boldsymbol{x})$	binary CE	GD, SGD, Newton/IRLS
softmax regression	$\boldsymbol{y} \in \{0, 1\}^K$	$\text{softmax}(\boldsymbol{W}\boldsymbol{x})$	categorical CE	GD, SGD, Newton

Table 4.1: The four single-layer networks of Sections 4.1–4.4. For many applications these networks are already sufficient — but they all have *linear* decision boundaries, which is their fundamental limitation (Section 4.5).

4.5 Single-Layer Networks Cannot Solve XOR

The lecture opens the multi-layer part with a quick task. Here are some labelled points — what are the missing labels?

$(-0.02,\ 0.01)$	$\to\ 0$	$(0.02,\ 0.98)$	$\to\ 1$
$(1.05,\ 0.99)$	$\to\ 0$	$(1.03,\ -0.02)$	$\to\ 1$
$(0.03,\ 0.01)$	$\to\ 0$	$(1.03,\ 1.01)$	$\to\ 0$
$(0.97,\ -0.01)$	$\to\ 1$	$(0.96,\ 0.03)$	$\to\ 1$
$(1.00,\ 0.01)$	$\to\ ?$	$(-0.04,\ 0.03)$	$\to\ ?$

The points cluster around the corners of the unit square, and the label is 1 exactly when one coordinate is (close to) 1 and the other (close to) 0: it is a noisy version of **XOR**. So the answers are 1 and 0. Easy for a human — and impossible for every network of the previous sections.

4.5.1 Why a Linear Network Fails

Take the four clean XOR points $\boldsymbol{x}^1 = (0, 0)$, $\boldsymbol{x}^2 = (1, 0)$, $\boldsymbol{x}^3 = (0, 1)$, $\boldsymbol{x}^4 = (1, 1)$ with labels $y^1 = 0$, $y^2 = 1$, $y^3 = 1$, $y^4 = 0$, and a linear (single-layer) network $g_1(\boldsymbol{x}; \boldsymbol{w}) = x_1 w_1 + x_2 w_2$. Fitting the labels exactly requires

$$0 = 0\,w_1 + 0\,w_2, \qquad 1 = 1\,w_1 + 0\,w_2, \qquad 1 = 0\,w_1 + 1\,w_2, \qquad 0 = 1\,w_1 + 1\,w_2.$$

The second and third equation give $w_1 = w_2 = 1$, and then the fourth says $0 = 2$. No parameters solve the problem.

The failure is more fundamental than this particular model. Adding a bias does not help, and neither does a sigmoid or softmax on the output: these are monotone functions of a *linear* score $z(\boldsymbol{x}) = \boldsymbol{w}^\mathsf{T}\boldsymbol{x} + b$, so the decision boundary is still a straight line. And no straight line separates XOR. For an affine function, $z(\boldsymbol{x}^2) + z(\boldsymbol{x}^3) = z(\boldsymbol{x}^1) + z(\boldsymbol{x}^4)$ (both sides equal $w_1 + w_2 + 2b$). The two class-1 points would need scores above the threshold and the two class-0 points scores below it — impossible when the two sums are equal.

4.5.2 A Hidden Layer Solves It

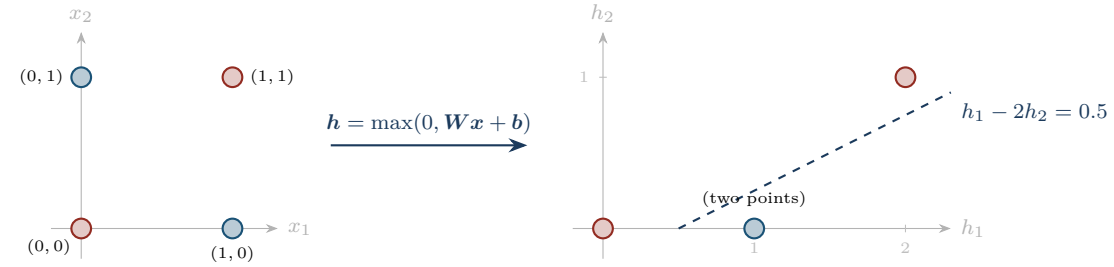
The way out is a hidden layer with a non-linearity. The lecture uses a two-layer network with ReLU hidden units:

$$g_2(\boldsymbol{x}; \boldsymbol{W}, \boldsymbol{w}, \boldsymbol{b}) = \boldsymbol{w}^\mathsf{T} \max(0, \boldsymbol{W}\boldsymbol{x} + \boldsymbol{b}) = \boldsymbol{w}^\mathsf{T}\boldsymbol{h}, \qquad \boldsymbol{W} = \begin{pmatrix} 1 & 1 \\ 1 & 1 \end{pmatrix}, \ \boldsymbol{b} = \begin{pmatrix} 0 \\ -1 \end{pmatrix}, \ \boldsymbol{w} = \begin{pmatrix} 1 \\ -2 \end{pmatrix}.$$

The XOR network, step by step

$\boldsymbol{x}$	$\boldsymbol{W}\boldsymbol{x} + \boldsymbol{b}$	$\boldsymbol{h} = \max(0, \boldsymbol{W}\boldsymbol{x} + \boldsymbol{b})$	$\boldsymbol{w}^\mathsf{T}\boldsymbol{h} = h_1 - 2h_2$	label
$(0, 0)$	$(0, -1)$	$(0, 0)$	0	0
$(1, 0)$	$(1, 0)$	$(1, 0)$	1	1
$(0, 1)$	$(1, 0)$	$(1, 0)$	1	1
$(1, 1)$	$(2, 1)$	$(2, 1)$	0	0

The network reproduces all four labels exactly. The first hidden unit counts how many inputs are on; the second fires only if both are on, and the output subtracts it twice.



input space:
no line separates the classes

hidden space:
linearly separable

Figure 4.4: What the hidden layer does. In input space (left) the XOR classes cannot be separated by a line. The hidden layer maps $(1, 0)$ and $(0, 1)$ to the same point $(1, 0)$ and $(1, 1)$ to $(2, 1)$. In this new space a single line — the output unit $h_1 - 2h_2$, thresholded at 0.5 — separates the classes.

Intuition & Mental Model

This is the key idea of all of deep learning in its smallest form: the hidden layer *transforms the data into a new representation* in which the problem becomes easy (here: linearly separable), and the output layer is just a linear model on top. In a deep network, every layer does this once more.

Universal function approximation (Hornik, 1989; Cybenko, 1989)

A multi-layer perceptron with a single hidden layer, enough hidden units and a suitable non-linear activation function (for example a sigmoid) can approximate any continuous function on a compact set to arbitrary accuracy.

So single-layer networks cannot solve XOR, a simple two-layer network with a ReLU non-linearity does, and MLPs can even approximate any continuous function. The theorem says that a solution *exists*; it says nothing about how many hidden units are needed or how to find the weights. Finding them is the job of backpropagation.

4.6 Multi-Layer Perceptron

4.6.1 Model and Forward Pass

A multi-layer perceptron (MLP) consists of an input layer L_1, one or more hidden layers, and an output layer; every unit of one layer is connected to every unit of the next. The lecture numbers the units consecutively through the network (Section 3.3): the D input units come first, the K output units are the last ones, $Q-K+1,\dots,Q$.

Quantities of an MLP

For every non-input unit i (i.e. $i > D$):

$$s_i = \sum_j w_{ij}\, a_j + b_i \qquad \text{(network input, pre-activation)}, \qquad a_i = f(s_i) \qquad \text{(activation)},$$

where the sum runs over all units j with a connection to i, and for input units $a_i = x_i$. The **architecture** of a network is given by the number of layers, the number of units per layer and the connections between them; the activation functions may also be counted as part of it. Different units can have different activation functions f_i.

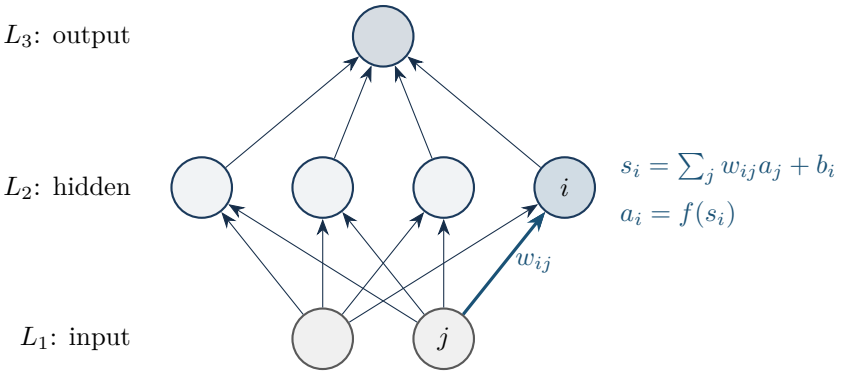


Figure 4.5: An MLP with one hidden layer. Unit i collects the weighted activations of the units below (pre-activation s_i, including its bias b_i) and passes them through its activation function.

Activation functions. The hidden units need a *non-linear* activation function — otherwise the whole network collapses to a linear model (Section 4.7). The lecture shows six common choices; Table 4.2 lists them with their derivatives (which backpropagation needs), and Fig. 4.6 plots them.

Name	$f(x)$	$f'(x)$
logistic sigmoid	$\sigma(x) = \dfrac{1}{1+e^{-x}}$	$\sigma(x)(1-\sigma(x)) \leq \frac{1}{4}$
tanh	$\tanh(x) = \dfrac{e^{x}-e^{-x}}{e^{x}+e^{-x}} = 2\sigma(2x)-1$	$1-\tanh^2(x) \leq 1$
ReLU	$\max(0, x)$	1 for $x > 0$, 0 for $x < 0$
leaky ReLU	x for $x > 0$, αx for $x \leq 0$ (small α)	1 for $x > 0$, α for $x < 0$
ELU	x for $x > 0$, $\alpha(e^{x}-1)$ for $x \leq 0$	1 for $x > 0$, αe^{x} for $x < 0$
SELU	λx for $x > 0$, $\lambda\alpha(e^{x}-1)$ for $x \leq 0$	λ for $x > 0$, $\lambda\alpha e^{x}$ for $x < 0$

Table 4.2: Common activation functions. For SELU, $\lambda \approx 1.0507$ and $\alpha \approx 1.6733$ are fixed constants (chosen so that activations stay normalized, see Chapter 10).

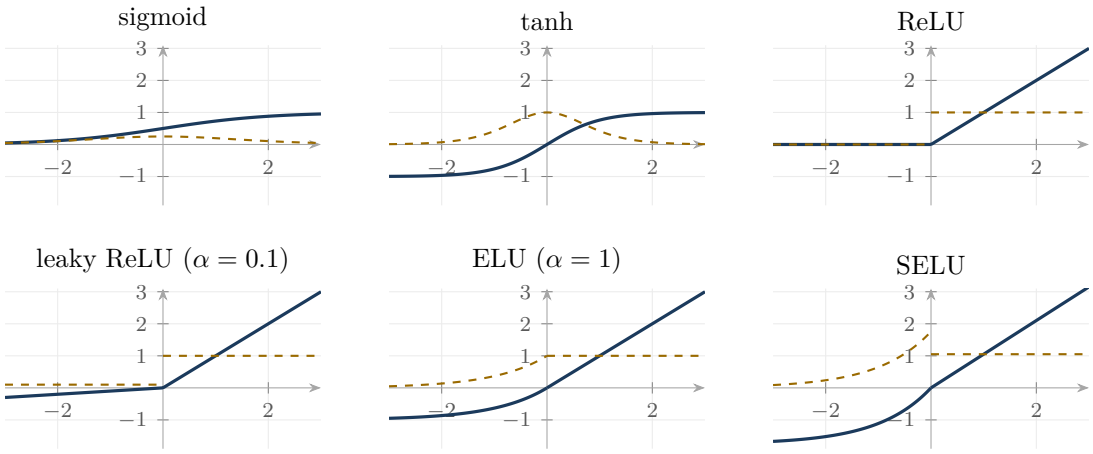


Figure 4.6: Activation functions (solid) and their derivatives (dashed). The sigmoid's derivative never exceeds 0.25 and both sigmoid and tanh saturate for large $|x|$. ReLU and its variants have derivative 1 (SELU: λ) for all positive inputs — the property that makes them attractive for deep networks (Section 4.7.3).

Output activations. The activation of the *output* units is chosen according to the task, exactly as in the single-layer networks: usually a **linear** activation for regression, the **logistic sigmoid** for binary (or multi-label) classification, and the **softmax** for multi-class classification. An MLP is therefore "a linear, logistic or softmax regression on top of learned hidden features".

Forward pass. Computing the output for an input is called the *forward pass*: start with the input units and go layer by layer upwards, computing s_i and a_i for every unit.

Algorithm 1: Forward pass of an MLP

Input: input $\boldsymbol{x}$, weights w_{ij}, layers $L_1, \dots, L_L$

for $i = 1, \dots, D$ **do** *// initialization*
  $a_i \leftarrow x_i$;
for $\nu = 2, \dots, L$ **do**
  for *all* $i \in L_\nu$ **do**
    $s_i \leftarrow \sum_{j:\, w_{ij} \text{ exists}} w_{ij}\, a_j$;
    $a_i \leftarrow f(s_i)$;

Output: $\hat{y}_i = (g(\boldsymbol{x}; \boldsymbol{w}))_i = a_i$ for all $Q - K + 1 \le i \le Q$

4.6.2 Gradient Descent and the Backpropagation Algorithm

To train an MLP we minimize the empirical error by gradient descent, just as before:

$$R_{\text{emp}}(\boldsymbol{w}, \boldsymbol{X}, \boldsymbol{Y}) = \frac{1}{N} \sum_{n=1}^{N} \mathcal{L}(\boldsymbol{y}^n, g(\boldsymbol{x}^n; \boldsymbol{w})), \qquad \boldsymbol{w}^{\text{new}} = \boldsymbol{w}^{\text{old}} - \eta \nabla_{\boldsymbol{w}} R_{\text{emp}},$$

where $g(\boldsymbol{x}^n; \boldsymbol{w})$ is the network and $\boldsymbol{w}$ contains all weights (weight matrices) of all layers and the biases. For a single data point the update uses $\nabla_{\boldsymbol{w}} \mathcal{L}(\boldsymbol{y}, g(\boldsymbol{x}; \boldsymbol{w}))$.

The only new problem is computing the gradient: a weight deep inside the network influences the loss through all the layers above it. **Backpropagation** — made popular by Rumelhart et al. (1986), proposed earlier by Werbos (1974), also called *delta propagation* — solves it with nothing but the chain rule, organized cleverly.

Step 1: split the derivative at the pre-activation. Weight w_{ij} influences the loss only through the pre-activation $s_i = \sum_j w_{ij} a_j + b_i$ of the unit it points to. By the chain rule,

$$\frac{\partial \mathcal{L}}{\partial w_{ij}} = \frac{\partial \mathcal{L}}{\partial s_i} \frac{\partial s_i}{\partial w_{ij}} = \frac{\partial \mathcal{L}}{\partial s_i}\, a_j, \qquad \text{since } \frac{\partial s_i}{\partial w_{ij}} = a_j.$$

Delta error

The **delta error** of unit i is the derivative of the loss with respect to its pre-activation:

$$\delta_i \coloneqq \frac{\partial}{\partial s_i} \mathcal{L}(\boldsymbol{y}^n, g(\boldsymbol{x}^n; \boldsymbol{w})).$$

With it, the gradient for every weight is simply

$$\frac{\partial \mathcal{L}}{\partial w_{ij}} = \delta_i\, a_j \qquad \text{(delta at the target unit} \times \text{activation at the source unit).}$$

Compare this with softmax regression: there, $\partial R_{\text{emp}} / \partial w_{ij} = (\text{softmax}(\boldsymbol{W}\boldsymbol{x})_i - y_i)\, x_j$, i.e. $\delta_i = a_i - y_i$ and $a_j = x_j$. The single-layer result is the special case of a network without hidden units.

Step 2: deltas at the output units. For an output unit k (with an element-wise activation),

$$\delta_k = \frac{\partial \mathcal{L}}{\partial a_k} f'(s_k), \qquad Q - K + 1 \le k \le Q,$$

where $a_k = (g(\boldsymbol{x}^n; \boldsymbol{w}))_k$ is the output activation and s_k the output pre-activation. For the matched loss/activation pairs this is the familiar error signal (Table 4.3) — the same as for linear, logistic and softmax regression.

Step 3: deltas at the hidden units (the recursion). A hidden unit j influences the loss only through the units i in the layer above that it is connected to, via $s_i = \sum_j w_{ij} f(s_j) + b_i$. The multivariate chain rule therefore gives

$$\delta_j = \frac{\partial \mathcal{L}}{\partial s_j} = \sum_i \frac{\partial \mathcal{L}}{\partial s_i} \frac{\partial s_i}{\partial s_j} = \sum_i \delta_i \, w_{ij} f'(s_j) = f'(s_j) \sum_i \delta_i \, w_{ij},$$

where the sum goes over all units i for which the weight w_{ij} exists.

Backpropagation (unit-wise)

$$\underbrace{\delta_k = \frac{\partial \mathcal{L}}{\partial a_k} f'(s_k),}_{\text{output units}} \qquad \underbrace{\delta_j = f'(s_j) \sum_i \delta_i \, w_{ij},}_{\text{hidden units}} \qquad \underbrace{\frac{\partial \mathcal{L}}{\partial w_{ij}} = \delta_i \, a_j, \quad \Delta w_{ij} = -\eta \, \delta_i \, a_j \,.}_{\text{gradient and update}}$$

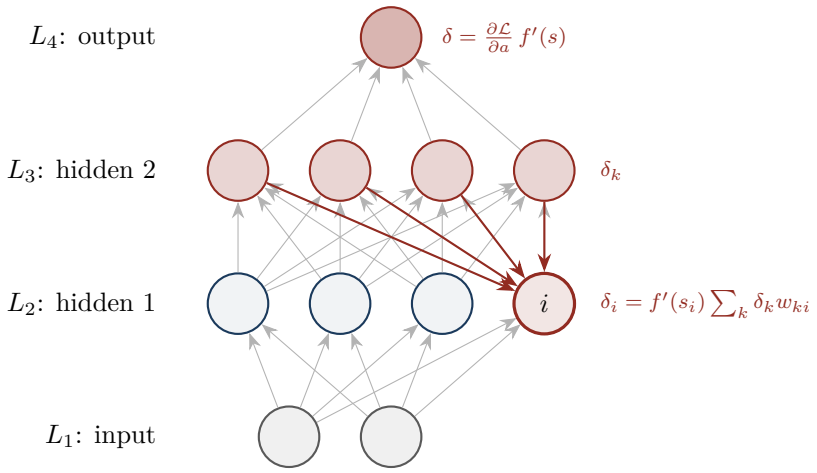


Figure 4.7: Delta propagation. The deltas are computed at the output and then passed *backwards* through the same weights that carried the activations forward: unit i in hidden layer 1 collects the deltas of all units k it feeds into, weighted by w_{ki}, and multiplies by its own $f'(s_i)$. The gradient for a weight w_{ki} is then $\delta_k a_i$.

Loss function		**Output activation**	**Delta error**
squared loss	$\frac{1}{2}(a-y)^2$	linear: $f(x) = x$	$\delta = a - y$
binary CE	$-y \log a - (1-y)\log(1-a)$	sigmoid: $f(x) = \frac{1}{1+e^{-x}}$	$\delta = a - y$
categorical CE	$-\sum_{k=1}^{K} y_k \log a_k$	softmax: $f(\boldsymbol{x}) = \text{softmax}(\boldsymbol{x})$	$\boldsymbol{\delta} = \boldsymbol{a} - \boldsymbol{y}$

Table 4.3: Frequently used loss functions and their matching output activations. For these pairs, the derivative of the activation cancels and the delta error at the output is simply "prediction minus target".

Potential problems of other combinations

The slides ask what goes wrong if loss and output activation do not match. Two examples:

- **Squared loss with a sigmoid output:** now $\delta = (a-y)\,\sigma'(s) = (a-y)\,a(1-a)$. If the network is *confidently wrong* (say $a \approx 0$ but $y = 1$), then $a(1-a) \approx 0$ and the delta almost vanishes — the network learns slowest exactly when it is most wrong. With cross-entropy the factor $a(1-a)$ cancels.
- **Cross-entropy with a linear output:** the output is not restricted to $(0, 1)$, so $\log a$ or $\log(1-a)$ may be undefined.

Also note that for the softmax the derivative is not element-wise: "$\partial\mathcal{L}/\partial a_k\, f'(s_k)$" has to be replaced by the full Jacobian of the softmax, which yields $\boldsymbol{\delta} = \boldsymbol{a} - \boldsymbol{y}$ in combination with the cross-entropy (Section 4.4).

Algorithm 2: Backward pass of an MLP (online version)

Input: activations a_i and pre-activations s_i from the forward pass, label $\boldsymbol{y}$, learning rate η

for $i = Q-K+1, \ldots, Q$ **do** *// output layer*
  $\delta_i \leftarrow \frac{\partial \mathcal{L}}{\partial a_i}\, f'(s_i)$;
  for *all* $j \in L_{L-1}$ **do**
    $\Delta w_{ij} \leftarrow -\eta\, \delta_i\, a_j$;
for $\nu = L-1, \ldots, 2$ **do** *// hidden layers, top to bottom*
  for *all* $i \in L_\nu$ **do**
    $\delta_i \leftarrow f'(s_i) \sum_k \delta_k\, w_{ki}$;
    for *all* $j \in L_{\nu-1}$ **do**
      $\Delta w_{ij} \leftarrow -\eta\, \delta_i\, a_j$;

Backpropagation in five steps (recap)

1. Select a sample $\boldsymbol{x}$ (randomly) from the training set and perform a forward pass. Memorize the activations a_i and pre-activations s_i of all units.
2. Compute the delta errors at the output units: $\delta_k = \frac{\partial \mathcal{L}}{\partial a_k} f'(s_k)$.
3. Compute the delta errors of the hidden layers, starting with the layer closest to the output, by backpropagating the error signal: $\delta_j = f'(s_j) \sum_i \delta_i w_{ij}$.
4. Compute the gradients $\frac{\partial \mathcal{L}}{\partial w_{ij}} = \delta_i a_j$.
5. Update the weights with a gradient-descent step: $\Delta w_{ij} = -\eta\, \delta_i\, a_j$.

Backpropagation with numbers (checked by finite differences)

A 2–2–1 network: sigmoid hidden units, a linear output unit, squared loss, no biases. Input $\boldsymbol{x} = (1, 0.5)^\mathsf{T}$, label $y = 1$,

$$\boldsymbol{W}^{[1]} = \begin{pmatrix} 0.5 & -0.3 \\ 0.2 & 0.8 \end{pmatrix}, \qquad \boldsymbol{W}^{[2]} = \begin{pmatrix} 0.7 & -0.4 \end{pmatrix}.$$

Forward pass. $\boldsymbol{s}^{[1]} = \boldsymbol{W}^{[1]}\boldsymbol{x} = (0.5 - 0.15,\ 0.2 + 0.4)^\mathsf{T} = (0.35, 0.60)^\mathsf{T}$, $\boldsymbol{a}^{[1]} = \sigma(\boldsymbol{s}^{[1]}) = (0.5866, 0.6457)^\mathsf{T}$, $\hat{y} = 0.7 \cdot 0.5866 - 0.4 \cdot 0.6457 = 0.1524$, loss $\mathcal{L} = \frac{1}{2}(0.1524 - 1)^2 = 0.3592$.

Output delta. Linear output with squared loss: $\delta^{[2]} = \hat{y} - y = -0.8476$.

Hidden deltas. $f'(\boldsymbol{s}^{[1]}) = \boldsymbol{a}^{[1]} \odot (1 - \boldsymbol{a}^{[1]}) = (0.2425, 0.2288)^\mathsf{T}$, so

$$\delta_1^{[1]} = 0.2425 \cdot 0.7 \cdot (-0.8476) = -0.1439, \qquad \delta_2^{[1]} = 0.2288 \cdot (-0.4) \cdot (-0.8476) = 0.0776.$$

Gradients (δ at the target times activation at the source):

$$\frac{\partial \mathcal{L}}{\partial \boldsymbol{W}^{[2]}} = \delta^{[2]}\, \boldsymbol{a}^{[1\mathsf{T}]} = (-0.4972,\ -0.5473), \qquad \frac{\partial \mathcal{L}}{\partial \boldsymbol{W}^{[1]}} = \boldsymbol{\delta}^{[1]} \boldsymbol{x}^\mathsf{T} = \begin{pmatrix} -0.1439 & -0.0719 \\ 0.0776 & 0.0388 \end{pmatrix}.$$

Central finite differences give the same six numbers (maximum deviation below 10^{-10}). One update with $\eta = 0.5$ raises the output to $\hat{y} = 0.4968$ and lowers the loss from 0.359 to 0.127.

Why backprop is efficient. The name refers to the fact that the delta errors of one layer are used to compute the deltas of the layer below: every delta is computed once and reused by all weights below it. For a network with W weights, a forward and a backward pass each cost $\mathcal{O}(W)$ operations, so the *whole* gradient costs $\mathcal{O}(W)$. Computing it naively by finite differences would need a separate forward pass for every single weight, i.e. $\mathcal{O}(W^2)$. A similar approach can be used to compute second derivatives.

Full batch, online and mini-batch learning. The gradient can be computed on the full training set before taking one step (*full-batch* gradient), on a single sample followed by an update (*online learning*), or on a small random subset of, e.g., 32 samples (*mini-batch training* or *stochastic gradient descent*). Details in Section 4.10.

4.6.3 Training MLPs

With MLPs we suddenly have many design choices: the number of layers, the number of neurons per layer, the activation function, the learning rate. These are **hyperparameters** — they are not learned by gradient descent. And minimizing the empirical error is not sufficient: what we care about is the error on *new* data (Chapter 5). So the architecture and hyperparameters are optimized on a separate *validation set*, and the final model is evaluated on a *test set* (Section 4.10).

The loss surface. For linear and logistic regression the error was convex. For an MLP the loss surface as a function of the weights is **highly non-convex**, with many local minima and saddle points. The minimum is also never unique, because of **weight-space symmetries**:

- For an odd activation such as tanh ($\tanh(-x) = -\tanh(x)$), flipping the signs of all weights into and out of a hidden unit gives exactly the same network function.
- The order of the hidden units within a layer can be permuted (along with their weights) without changing the function.

A layer with M tanh units therefore has $M!\,2^M$ equivalent weight settings for every solution. Despite all this, there are theoretical results indicating that SGD still finds "good" minima (more in the follow-up course).

4.7 (Deep) Feed-Forward Neural Networks

Deep feed-forward networks (DNNs) work on the same principle as MLPs — layers of interconnected neurons — but with more layers, more neurons, and different activation functions. The boundary between the two is informal (Table 4.4).

	MLP	**DNN**
number of hidden layers	1 or few	> 1, up to several hundreds
number of neurons per layer	< 10	> 100
activation function	tanh, sigmoid	ReLU, SELU

Table 4.4: Typical MLPs versus deep feed-forward networks (informal guidelines from the lecture).

For deep networks the unit-by-unit numbering becomes unwieldy, so we switch to the **layer-wise notation** of Section 3.4: the input is $\boldsymbol{a}^{[0]} = \boldsymbol{x}$, layer l has weight matrix $\boldsymbol{W}^{[l]}$, pre-activations $\boldsymbol{s}^{[l]}$ and activations $\boldsymbol{a}^{[l]}$, and the activation function f is applied element-wise. (In the MLP picture the units are numbered $1, \dots, Q$ snaking through the network; in the DNN picture each layer is numbered separately.)

4.7.1 Forward Pass

From one layer to the next:

$$\boldsymbol{a}^{[l]} = f(\boldsymbol{s}^{[l]}) = f(\boldsymbol{W}^{[l]}\boldsymbol{a}^{[l-1]}) \qquad \text{or, with bias units,} \qquad \boldsymbol{a}^{[l]} = f(\boldsymbol{W}^{[l]}\boldsymbol{a}^{[l-1]} + \boldsymbol{b}^{[l]}).$$

The whole network is a nested function,

$$\hat{\boldsymbol{y}} = g(\boldsymbol{x}; \boldsymbol{W}^{[1]}, \dots, \boldsymbol{W}^{[L]}) = \sigma\Big(\boldsymbol{W}^{[L]}(\cdots f(\boldsymbol{W}^{[2]} f(\boldsymbol{W}^{[1]}\boldsymbol{x}))\cdots)\Big),$$

with the output activation σ (linear, sigmoid or softmax) chosen for the task.

Why non-linearities?. Without the non-linear f in the hidden layers, the nested function collapses:

$$g(\boldsymbol{x}; \boldsymbol{W}^{[1]}, \dots, \boldsymbol{W}^{[L]}) = \sigma(\underbrace{\boldsymbol{W}^{[L]} \cdots \boldsymbol{W}^{[1]}}_{=\hat{\boldsymbol{W}}}\, \boldsymbol{x}) = \sigma(\hat{\boldsymbol{W}}\boldsymbol{x}).$$

A product of matrices is just another matrix, so a deep *linear* network is no more powerful than a single layer — it cannot even solve XOR. Depth only pays off with non-linear activations.

4.7.2 Backpropagation in DNNs

Without loss of generality, take a weight in the *first* layer, the one furthest from the loss. The chain rule through all layers gives a product of Jacobians:

$$\frac{\partial}{\partial w_{ij}^{[1]}} \mathcal{L}(\boldsymbol{y}, g(\boldsymbol{x}; \boldsymbol{W}^{[1]}, \dots, \boldsymbol{W}^{[L]})) = \underbrace{\frac{\partial \mathcal{L}(\boldsymbol{y}, \hat{\boldsymbol{y}})}{\partial \hat{\boldsymbol{y}}} \frac{\partial \hat{\boldsymbol{y}}}{\partial \boldsymbol{s}^{[L]}}}_{=: A} \cdots \underbrace{\frac{\partial \boldsymbol{s}^{[l]}}{\partial \boldsymbol{s}^{[l-1]}}}_{=: B} \cdots \underbrace{\frac{\partial \boldsymbol{s}^{[1]}}{\partial w_{ij}^{[1]}}}_{=: C}.$$

(Derivatives of scalars with respect to vectors are row vectors here, so the product is read left to right.) We treat the three kinds of factors in turn, after defining the layer-wise delta errors as the transposed derivative of the loss with respect to the pre-activations:

$$\boldsymbol{\delta}^{[l\mathsf{T}]} := \frac{\partial \mathcal{L}(\boldsymbol{y}, \hat{\boldsymbol{y}})}{\partial \boldsymbol{s}^{[l]}}.$$

A — the output layer. A depends on the choice of loss and output activation. For the matched pairs of Table 4.3 (canonical links) it is simply

$$\boldsymbol{\delta}^{[L\mathsf{T}]} = A = (\hat{\boldsymbol{y}} - \boldsymbol{y})^{\mathsf{T}}.$$

B — from one layer to the one below. Since $\boldsymbol{s}^{[l]} = \boldsymbol{W}^{[l]} f(\boldsymbol{s}^{[l-1]})$, the Jacobian of one layer's pre-activations with respect to the previous layer's is

$$\frac{\partial \boldsymbol{s}^{[l]}}{\partial \boldsymbol{s}^{[l-1]}} = \boldsymbol{W}^{[l]} \operatorname{diag}(f'(\boldsymbol{s}^{[l-1]})) \coloneqq \boldsymbol{J}^{[l]}.$$

This gives the recursive formula for the deltas:

$$\boldsymbol{\delta}^{[l-1\mathsf{T}]} = \frac{\partial \mathcal{L}}{\partial \boldsymbol{s}^{[l]}} \frac{\partial \boldsymbol{s}^{[l]}}{\partial \boldsymbol{s}^{[l-1]}} = \boldsymbol{\delta}^{[l\mathsf{T}]} \boldsymbol{W}^{[l]} \operatorname{diag}(f'(\boldsymbol{s}^{[l-1]})) = \boldsymbol{\delta}^{[l\mathsf{T}]} \boldsymbol{J}^{[l]},$$

or, transposed and written element-wise,

$$\boldsymbol{\delta}^{[l-1]} = f'(\boldsymbol{s}^{[l-1]}) \odot (\boldsymbol{W}^{[l\mathsf{T}]} \boldsymbol{\delta}^{[l]}).$$

The second form is the matrix version of $\delta_j = f'(s_j) \sum_i \delta_i w_{ij}$ from the MLP.

C — the weight itself. $\boldsymbol{s}^{[1]} = \boldsymbol{W}^{[1]} \boldsymbol{a}^{[0]}$, and $w_{ij}^{[1]}$ only appears in component i:

$$\frac{\partial \boldsymbol{s}^{[1]}}{\partial w_{ij}^{[1]}} = (0, \ldots, 0, a_j^{[0]}, 0, \ldots, 0)^{\mathsf{T}} \quad \text{(non-zero entry at position } i\text{)}.$$

Putting it together. For any layer l,

$$\frac{\partial \mathcal{L}}{\partial w_{ij}^{[l]}} = \frac{\partial \mathcal{L}}{\partial \boldsymbol{s}^{[l]}} \frac{\partial \boldsymbol{s}^{[l]}}{\partial w_{ij}^{[l]}} = (\boldsymbol{\delta}^{[l]})_i \, (\boldsymbol{a}^{[l-1]})_j, \qquad \text{or as an outer product:} \qquad \frac{\partial \mathcal{L}}{\partial \boldsymbol{W}^{[l]}} = \boldsymbol{\delta}^{[l]} \, \boldsymbol{a}^{[l-1\mathsf{T}]}.$$

Backpropagation in deep networks

1. Compute the deltas at the output: $\boldsymbol{\delta}^{[L]}$ (e.g. $\hat{\boldsymbol{y}} - \boldsymbol{y}$).
2. Backpropagate them through the network:
$$\boldsymbol{\delta}^{[l-1\mathsf{T}]} = \boldsymbol{\delta}^{[l\mathsf{T}]} \underbrace{\boldsymbol{W}^{[l]} \operatorname{diag}(f'(\boldsymbol{s}^{[l-1]}))}_{\boldsymbol{J}^{[l]}}.$$
3. Compute the weight updates: $\Delta \boldsymbol{W}^{[l]} = -\eta \, \boldsymbol{\delta}^{[l]} \, \boldsymbol{a}^{[l-1\mathsf{T}]}$.

Shape check: $\delta\, a^{\mathsf{T}}$, not $a\, \delta^{\mathsf{T}}$

The gradient with respect to $\boldsymbol{W}^{[l]} \in \mathbb{R}^{n_l \times n_{l-1}}$ must have the same shape as $\boldsymbol{W}^{[l]}$. Since $\boldsymbol{\delta}^{[l]} \in \mathbb{R}^{n_l}$ and $\boldsymbol{a}^{[l-1]} \in \mathbb{R}^{n_{l-1}}$, the outer product is $\boldsymbol{\delta}^{[l]} \boldsymbol{a}^{[l-1\mathsf{T}]}$ — entry (i, j) is $\delta_i a_j$. One slide writes $\boldsymbol{a}^{[l-1]} \boldsymbol{\delta}^{[l\mathsf{T}]}$, which is the transpose; the update rule on the following slide uses the correct $\boldsymbol{\delta}^{[l]} (\boldsymbol{a}^{[l-1]})^{\mathsf{T}}$. When in doubt, check the shape.

4.7.3 The Vanishing Gradient Problem

The recursion $\boldsymbol{\delta}^{[l-1\mathsf{T}]} = \boldsymbol{\delta}^{[l\mathsf{T}]}\boldsymbol{J}^{[l]}$ reveals the central problem of deep networks. Unrolled from the output down to the first layer,

$$\boldsymbol{\delta}^{[1\mathsf{T}]} = \boldsymbol{\delta}^{[L\mathsf{T}]}\,\boldsymbol{J}^{[L]}\boldsymbol{J}^{[L-1]}\cdots\boldsymbol{J}^{[2]},$$

the delta errors are multiplied by one Jacobian per layer. If the norms of all these Jacobians are bounded by some k, $\left\|\boldsymbol{J}^{[l]}\right\| \leq k$, then

$$\left\|\boldsymbol{\delta}^{[1]}\right\| \leq k^{L-1}\,\left\|\boldsymbol{\delta}^{[L]}\right\|.$$

The size of the error signal changes *exponentially* with depth. For $k < 1$ it vanishes: the lower layers receive almost no gradient and do not learn.

Vanishing and exploding gradients (Hochreiter, 1991)

Because the deltas are multiplied by the Jacobian $\boldsymbol{J}^{[l]} = \boldsymbol{W}^{[l]}\,\mathrm{diag}(f'(\boldsymbol{s}^{[l-1]}))$ in every layer, their norm grows or shrinks exponentially across layers. Three regimes:

- $\left\|\boldsymbol{\delta}^{[l-1]}\right\| < \left\|\boldsymbol{\delta}^{[l]}\right\|$: **vanishing gradients.** The typical case for sigmoid networks: since $|\sigma'(x)| \leq 0.25$, $\left\|\mathrm{diag}(f'(\boldsymbol{s}^{[l-1]}))\right\| \leq 0.25$, and the deltas shrink in every layer unless the weights compensate.
- $\left\|\boldsymbol{\delta}^{[l-1]}\right\| \approx \left\|\boldsymbol{\delta}^{[l]}\right\|$: **stable gradients.** The ideal case; many algorithmic improvements (e.g. initialization strategies, Chapter 9) aim at keeping this ratio close to one.
- $\left\|\boldsymbol{\delta}^{[l-1]}\right\| > \left\|\boldsymbol{\delta}^{[l]}\right\|$: **exploding gradients.** If the weight matrices have large norm, the deltas grow through the layers; huge weight updates make learning unstable or cause numerical overflows.

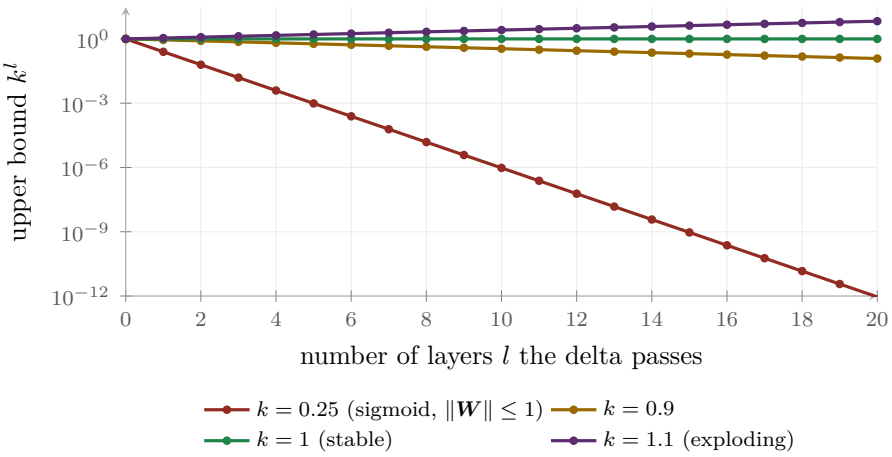


Figure 4.8: How much of the error signal survives l layers, for different per-layer factors k (log scale). With the sigmoid's factor 0.25, ten layers already shrink the delta by a factor of about $4 \cdot 10^{-6}$.

How fast do gradients vanish?

With sigmoid units and weight matrices of norm at most one, $k = 0.25$. The delta at the first layer of a network with L layers is then at most 0.25^{L-1} times the output delta:

$$L = 5: \ 0.25^4 \approx 3.9 \cdot 10^{-3}, \quad L = 10: \ 0.25^9 \approx 3.8 \cdot 10^{-6}, \quad L = 20: \ 0.25^{19} \approx 3.6 \cdot 10^{-12}.$$

In a 20-layer sigmoid network, the first layers would learn about a trillion times more slowly than the last one.

Ameliorations for feed-forward networks (outlook). The lecture names several remedies, each covered later:

- **rectified linear units** — $f'(x) = 1$ for all positive inputs, so the activation function does not shrink the delta;
- **initialization strategies** that keep the norm of the Jacobians close to one (Section 4.10 and Chapter 9);
- **skip connections** (ResNets), which give the gradient a shortcut around layers;
- and more (e.g. normalization, Chapter 10). For recurrent networks, LSTM is the classic solution (Chapter 1).

4.8 Jacobian

"The Jacobian of a neural network" is an ambiguous term. Depending on what is differentiated with respect to what, it means one of three things:

Three Jacobians of a neural network

1. **Parameter Jacobian** — the derivative of the loss with respect to the parameters, $\dfrac{\partial \mathcal{L}(\boldsymbol{y}, g(\boldsymbol{x}; \boldsymbol{w}))}{\partial \boldsymbol{w}}$. This is the gradient used to train the network by gradient descent (computed by backpropagation).
2. **Input–output Jacobian** — the derivative of the network outputs with respect to the inputs, $\dfrac{\partial g(\boldsymbol{x}; \boldsymbol{w})}{\partial \boldsymbol{x}} = \dfrac{\partial \hat{\boldsymbol{y}}}{\partial \boldsymbol{x}}$. It tells how sensitive the prediction is to each input feature and can be used to optimize the *input* with respect to the output, by gradient *ascent* on $\boldsymbol{x}$ with fixed weights. "Deep Dream" images are made this way: an image is changed so that it excites certain units more and more (a horizon turns into towers and pagodas, leaves into birds and insects).
3. **(Single-layer) Jacobian** — the derivative of the outputs of one layer with respect to that layer's inputs. For $\boldsymbol{a} = f(\boldsymbol{W}\boldsymbol{x})$:

$$\boldsymbol{J} = \frac{\partial \boldsymbol{a}}{\partial \boldsymbol{x}} = \operatorname{diag}(f'(\boldsymbol{W}\boldsymbol{x}))\, \boldsymbol{W}.$$

This is the quantity connected to the vanishing gradient problem; when people say "the Jacobian" without further qualification, they usually mean this one.

Intuition & Mental Model

The single-layer Jacobian $\operatorname{diag}(f'(\boldsymbol{W}\boldsymbol{x}))\,\boldsymbol{W}$ and the factor $\boldsymbol{J}^{[l]} = \boldsymbol{W}^{[l]} \operatorname{diag}(f'(\boldsymbol{s}^{[l-1]}))$ of the backprop recursion are built from the same two ingredients — a weight matrix and the activation derivatives — just chained in a different order (one goes from activations to activations, the other from pre-activations to pre-activations). Either way: backpropagation multiplies by one such matrix per layer, and its norm decides whether gradients vanish, stay stable or explode.

4.9 Hessian

The Hessian is the matrix of second derivatives of the loss with respect to the weights, where all parameters of the network are written as one long vector $\boldsymbol{w}$:

$$H_{ij} = \frac{\partial^2 \mathcal{L}(\boldsymbol{y}, g(\boldsymbol{x}; \boldsymbol{w}))}{\partial w_i \, \partial w_j}.$$

It describes the *curvature* of the loss surface. Its main use is the second-order Taylor approximation of the error around a point $\boldsymbol{w}_0$, with gradient $\boldsymbol{g} = \nabla_{\boldsymbol{w}} R(\boldsymbol{w}_0)$:

$$R(\boldsymbol{w}) = R(\boldsymbol{w}_0) + (\boldsymbol{w} - \boldsymbol{w}_0)^\mathsf{T} \boldsymbol{g} + \frac{1}{2}(\boldsymbol{w} - \boldsymbol{w}_0)^\mathsf{T} \boldsymbol{H} (\boldsymbol{w} - \boldsymbol{w}_0) + \mathcal{O}\Big(\|\boldsymbol{w} - \boldsymbol{w}_0\|^3\Big).$$

The lecture lists its applications (covered later): **optimization** (Newton-type methods — with a question mark, see below), **pruning** (removing the weights whose removal increases the error least), **error bars** for predictions, and the **second-order Taylor approximation** itself.

Why not just use Newton's method for deep networks?

For logistic regression, the Hessian is a $D \times D$ matrix and Newton's method (IRLS) is the standard (Section 4.3). A deep network with W weights has a $W \times W$ Hessian: for a million weights that is 10^{12} entries, and inverting it costs $\mathcal{O}(W^3)$. In addition, the loss of a deep network is non-convex, so the Hessian is not positive definite in general, and a Newton step can even go uphill. This is why first-order methods (SGD and its variants) dominate deep learning.

4.10 Efficient Training: Basic Tricks of the Trade

The last section collects the practical knowledge needed to actually train an MLP or deep network: how to split the data, how to feed it to the network, how to prepare the inputs, how to initialize the weights and how to choose the learning rate and the network size.

4.10.1 Training, Validation and Test Set

Training, validation and test set ("sloppy" definitions)

- **Training set** (e.g. 80 % of the data): the samples that are presented to the network; the *weights* are adjusted to fit this set.
- **Validation set** (e.g. 10 %): withheld from the network; it is used to check the network during development, and the *architecture and hyperparameters* are adjusted on this set.

- **Test set** (e.g. 10 %): withheld from the network *and* from architecture and hyperparameter selection; it is used only to check the final trained network.

Precise definitions follow in Chapter 5.

Why a separate test set?

Every decision made by looking at a data set (stopping epoch, learning rate, number of layers, ...) slightly fits the model to that data set. After tuning on the validation set, the validation error is therefore an *optimistic* estimate of the error on new data. Only a test set that was never used for any decision gives an unbiased estimate.

4.10.2 Online, Stochastic and Batch Training

- **Online learning:** a single sample is propagated through the network, then the parameters are updated: $\boldsymbol{w}^{\text{new}} = \boldsymbol{w}^{\text{old}} - \eta \nabla_{\boldsymbol{w}} R_{\text{emp}}(y, \boldsymbol{x}, \boldsymbol{w})|_{\boldsymbol{w}^{\text{old}}}$.
- **Full-batch learning:** all training samples are propagated through the network, then one update is performed: $\boldsymbol{w}^{\text{new}} = \boldsymbol{w}^{\text{old}} - \eta \nabla_{\boldsymbol{w}} R_{\text{emp}}(\boldsymbol{y}, \boldsymbol{X}, \boldsymbol{w})|_{\boldsymbol{w}^{\text{old}}}$.
- **Stochastic (mini-batch) learning:** a small random subset is propagated, then the parameters are updated with the mini-batch estimate $\tilde{\nabla}_{\boldsymbol{w}}$ of the full-batch gradient:

$$\nabla_{\boldsymbol{w}} R_{\text{emp}}(\boldsymbol{y}, \boldsymbol{X}, \boldsymbol{w}) = \frac{1}{N} \sum_{n=1}^{N} \nabla_{\boldsymbol{w}} \mathcal{L}(y^n, g(\boldsymbol{x}^n; \boldsymbol{w})) \;\approx\; \frac{1}{B} \sum_{b=1}^{B} \nabla_{\boldsymbol{w}} \mathcal{L}(y^{n_b}, g(\boldsymbol{x}^{n_b}; \boldsymbol{w})) = \tilde{\nabla}_{\boldsymbol{w}} R_{\text{emp}}.$$

Intuition & Mental Model

Full-batch gradients are exact but expensive: one step costs a pass over the whole data set. Online updates are cheap but very noisy. Mini-batches are the compromise: the gradient estimate is good enough to make progress, many updates fit into one pass over the data, and a batch of, say, 32 samples is processed in parallel on a GPU about as fast as a single one. The noise even helps a little to escape poor regions of the loss surface.

4.10.3 Epochs, Batch Size and Learning Curves

Typically, mini-batches are drawn randomly *without replacement* from the training set. When the training set is used up, a new **epoch** starts; one epoch means that every training sample has been propagated through the network once. With N samples and batch size B, an epoch consists of about N/B updates.

Training is monitored with **learning curves**:

- after each mini-batch: the loss on the training (mini-)batch, optionally other metrics such as accuracy;
- after each epoch (or after a fixed number of updates): the loss and other metrics on the validation set.

The lecture's example is a DenseNet trained on CIFAR-100 for 300 epochs: training and test error drop quickly at first, then flatten out; at epoch 150 both drop sharply once more (at that point the learning rate is decreased, a common schedule); afterwards the training error goes to almost zero while the test error stays at about 0.26. The gap between the two curves is a first glimpse of *overfitting* (Chapter 5).

4.10.4 Normalizing Inputs and Outputs

- **Inputs should be normalized to zero mean and unit variance**: compute mean μ_d and standard deviation s_d of every feature d *on the training set*, and transform $x_d \mapsto (x_d - \mu_d)/s_d$. Training is faster this way. With unnormalized inputs, pre-activations can be large, sigmoid/tanh units start in their saturated region where the gradient is small; with normalized inputs they start near zero, where the gradient is large.
- **Categorical features** are one-hot encoded (Section 4.4).

Use the training statistics everywhere

The normalization constants μ_d, s_d are computed on the training set only and then applied unchanged to validation and test data (and to new data in deployment). Computing them on the full data set leaks information from the test set into training.

4.10.5 Initialization

- Weights should be initialized randomly with **zero mean**, from a Gaussian, uniform or truncated Gaussian distribution.
- **Variance:** $2/J$ for ReLU activations and $1/J$ otherwise, where J is the number of inputs of the unit (its *fan-in*, the number of units in the layer below).

The scaling with $1/J$ keeps the variance of the pre-activations roughly constant from layer to layer — which, by the argument of Section 4.7.3, also keeps the delta errors from vanishing or exploding at the start of training. The factor 2 for ReLU compensates for the fact that a ReLU sets about half of its inputs to zero. Details in Chapter 9.

Why not initialize all weights with zero?

If all weights of a layer start with the same value, all hidden units of the layer compute the same function and receive the same delta error — they stay identical forever, and the layer behaves like a single unit. The random initialization is needed to *break the symmetry*.

4.10.6 Learning Rates

The learning rate is one of the most important hyperparameters. Typical values are 0.1 or 0.01; it is searched on a logarithmic grid $10^{-1}, 10^{-2}, 10^{-3}, 10^{-4}, \ldots$ A useful heuristic: start training with a learning rate as high as possible without the network diverging, and decrease it later (as in the DenseNet learning curve above).

4.10.7 Number of Neurons and Hidden Layers

How large the network has to be depends on the complexity of the problem: more complex problems need more layers and more neurons. The lecture's **rule of thumb**: use as many layers and neurons as needed such that the network can *overfit* the training data. Then add regularization (Chapter 8) to prevent overfitting.

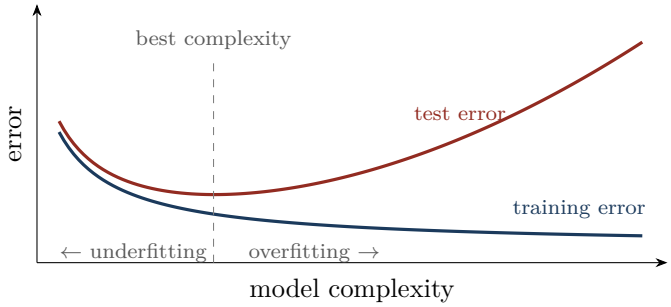


Figure 4.9: Outlook on generalization (schematic). The training error keeps decreasing as the model becomes more complex, but the error on new (test) data first decreases and then increases again: too complex a model fits the noise of the training set. Chapter 5 makes this precise.

4.11 Excursion: Amari–Hopfield Networks

The single-layer lecture closes with a different kind of network. Amari–Hopfield networks are **associative memory** systems: they store a set of patterns and, given a noisy or incomplete version of one of them, retrieve the complete pattern — *content-based retrieval*. They are similar to the Lenz–Ising models of physics, but learnable; Hopfield received the 2024 Nobel Prize in Physics for them (J. J. Hopfield, 1982), and he was criticized by J. Schmidhuber for not crediting S.-I. Amari, who had described such networks in 1972.

Amari–Hopfield network

- **Binary patterns:** inputs and outputs are vectors $\boldsymbol{x} = (x_1, \ldots, x_D) \in \{-1, 1\}^D$; the outputs are fed back as new inputs, so the network is *recurrent*.
- **Stored patterns:** $\boldsymbol{x}^1, \ldots, \boldsymbol{x}^N$.
- **Learning (Hebbian rule):**

$$\boldsymbol{W} = \frac{1}{N}\Big(\sum_{n=1}^{N} \boldsymbol{x}^n \boldsymbol{x}^{n\mathsf{T}}\Big) - \boldsymbol{I}_D.$$

- **Retrieval (inference):** starting from a noisy pattern, update

$$\boldsymbol{x}^{\text{new}} = \operatorname{sign}(\boldsymbol{W}\boldsymbol{x}^{\text{old}} - \boldsymbol{b}).$$

- **Energy:** the network is energy-based, with

$$E = -\frac{1}{2}\boldsymbol{x}^{\mathsf{T}}\boldsymbol{W}\boldsymbol{x} + \boldsymbol{x}^{\mathsf{T}}\boldsymbol{b} = -\frac{1}{2}\sum_{i=1}^{D}\sum_{j=1}^{D} w_{ij}x_i x_j + \sum_{i=1}^{D} b_i x_i.$$

  Stored patterns are (local) minima of the energy; retrieval runs "downhill".

Intuition & Mental Model

The Hebbian rule is Hebb's "fire together, wire together" in formula form: w_{ij} grows when units i and j have the same sign in many stored patterns ($x_i^n x_j^n = +1$) and shrinks when they have opposite signs. Since $x_i^2 = 1$, every outer product $\boldsymbol{x}^n\boldsymbol{x}^{n\mathsf{T}}$ has ones on the diagonal, so $\frac{1}{N}\sum_n \boldsymbol{x}^n\boldsymbol{x}^{n\mathsf{T}}$ has a diagonal of exactly one — subtracting $\boldsymbol{I}_D$ removes the self-connections. Each unit is then pushed towards the sign that the *other* units "vote"

for, given the correlations stored in $\boldsymbol{W}$.

Retrieving a pattern with one flipped bit

Store two orthogonal patterns in a network with $D = 8$ units,

$$\boldsymbol{x}^1 = (1,1,1,1,-1,-1,-1,-1), \qquad \boldsymbol{x}^2 = (1,-1,1,-1,1,-1,1,-1),$$

with $\boldsymbol{W} = \frac{1}{2}(\boldsymbol{x}^1\boldsymbol{x}^{1\mathsf{T}} + \boldsymbol{x}^2\boldsymbol{x}^{2\mathsf{T}}) - \boldsymbol{I}_8$ and $\boldsymbol{b} = \boldsymbol{0}$. Present $\boldsymbol{x}^1$ with its first bit flipped: $\boldsymbol{x}^{\text{old}} = (-1,1,1,1,-1,-1,-1,-1)$.

- $\boldsymbol{W}\boldsymbol{x}^{\text{old}} = (3,3,1,3,-3,-1,-3,-1)$, so $\text{sign}(\boldsymbol{W}\boldsymbol{x}^{\text{old}}) = (1,1,1,1,-1,-1,-1,-1) = \boldsymbol{x}^1$: the pattern is retrieved in one step.
- The energy drops from $E = -6$ for the noisy pattern to $E = -12$ for $\boldsymbol{x}^1$ (and $\boldsymbol{x}^1$, $\boldsymbol{x}^2$ are both fixed points of the update).

The same happens for every single flipped bit of $\boldsymbol{x}^1$.

4.12 Chapter Checklist

What you should take away from this chapter:

- the four single-layer networks (Table 4.1): model, loss, and how the loss follows from maximum likelihood (Gaussian noise → squared loss, Bernoulli → binary cross-entropy, categorical → categorical cross-entropy);
- the least-squares solution $\hat{\boldsymbol{w}} = (\boldsymbol{X}^\mathsf{T}\boldsymbol{X})^{-1}\boldsymbol{X}^\mathsf{T}\boldsymbol{y}$, gradient descent, and mini-batch SGD;
- the perceptron learning rule, its geometric meaning, and the convergence theorem with its conditions (linear separability; solution not unique; no convergence on non-separable data);
- $\sigma' = \sigma(1-\sigma) \leq 0.25$; the gradient and Hessian of logistic regression; Newton's method and IRLS; why one Newton step solves linear regression;
- softmax, one-hot encoding, and the common gradient form (prediction − target) × input;
- why single-layer networks cannot solve XOR and how a hidden layer with a non-linearity does; the universal approximation theorem;
- the MLP quantities s_i, a_i, f; common activation functions and their derivatives; matching output activations and losses;
- backpropagation: the delta errors, the output and hidden-layer formulas, $\partial\mathcal{L}/\partial w_{ij} = \delta_i a_j$, the matrix form $\boldsymbol{\delta}^{[l-1]} = f'(\boldsymbol{s}^{[l-1]}) \odot (\boldsymbol{W}^{[l\mathsf{T}]}\boldsymbol{\delta}^{[l]})$ and $\partial\mathcal{L}/\partial\boldsymbol{W}^{[l]} = \boldsymbol{\delta}^{[l]}\boldsymbol{a}^{[l-1\mathsf{T}]}$; its cost $\mathcal{O}(W)$;
- why deep networks need non-linearities; the vanishing/exploding gradient problem and its remedies;
- the three meanings of "Jacobian", the Hessian and its uses;
- the tricks of the trade: train/validation/test split, online vs. mini-batch vs. full-batch, epochs and learning curves, input normalization, initialization variance $2/J$ or $1/J$, learning-rate heuristics, sizing the network;
- Amari–Hopfield networks: Hebbian rule, energy, retrieval.

5

Generalization Error, Test Set and Cross-Validation

Minimizing the empirical error is not the actual goal — a network that memorizes its training data can still be useless on new data. This chapter makes precise what "performing well on new data" means, how to estimate it honestly, and how to use these estimates to choose models and hyperparameters.

Planned content: Slides not yet available. Expected structure below; adapt it to the slides.

5.1 Empirical Error and Generalization Error

Planned content: Data-generating distribution; expected loss (generalization error, risk) R_{gen} versus empirical error R_{emp}; why minimizing R_{emp} is only a proxy.

5.2 Overfitting and Underfitting

Planned content: Model complexity, capacity; training versus test error (Fig. 4.9); bias–variance decomposition if covered.

5.3 Estimating the Generalization Error: Test Set

Planned content: Precise definitions of training, validation and test set (refining Section 4.10); unbiasedness of the test-set estimate; variance of the estimate and test-set size.

5.4 Cross-Validation

Planned content: k-fold cross-validation, leave-one-out; using CV for hyperparameter selection; nested CV; pitfalls (information leakage).

5.5 Chapter Checklist

Planned content: To be written together with the chapter.

6

Convolutional Neural Networks

Fully connected networks ignore the structure of their input: for them, an image is just a long vector, and a cat in the top-left corner has nothing to do with a cat in the bottom-right. Convolutional networks build the spatial structure into the architecture (Section 1.5).

Planned content: Slides not yet available. Expected structure below; adapt it to the slides.

6.1 Convolution and Cross-Correlation

Planned content: Discrete convolution in 1D and 2D; kernels/filters; padding and stride; output-size formula; channels.

6.2 Convolutional Layers

Planned content: Local connectivity and weight sharing; number of parameters compared with a fully connected layer; feature maps; translation equivariance.

6.3 Pooling

Planned content: Max and average pooling; translation invariance; downsampling.

6.4 CNN Architectures and Training

Planned content: Typical layer stacks (convolution → non-linearity → pooling, fully connected head); backpropagation through convolution and pooling; historical architectures (Neocognitron, LeNet, AlexNet) as covered.

6.5 Chapter Checklist

Planned content: To be written together with the chapter.

7

Learning Methods: Basic Algorithms

Chapter 4 trained everything with plain (stochastic) gradient descent and mentioned Newton's method. This chapter looks at the optimization algorithms actually used to train deep networks.

Planned content: Slides not yet available. Expected structure below; adapt it to the slides.

7.1 Gradient Descent Revisited

Planned content: Batch, mini-batch and online gradient descent (Section 4.10); convergence and the role of the learning rate.

7.2 Momentum and Adaptive Learning Rates

Planned content: Momentum, Nesterov momentum; per-parameter learning rates (AdaGrad, RMSProp, Adam) as covered.

7.3 Learning-Rate Schedules

Planned content: Step decay (cf. the DenseNet learning curve in Section 4.10), warm-up, cosine schedules as covered.

7.4 Second-Order Methods

Planned content: Newton's method (Section 4.3), why it does not scale to deep networks (Section 4.9), approximations if covered.

7.5 Chapter Checklist

Planned content: To be written together with the chapter.

8

Regularization

The rule of thumb of Section 4.10 was: make the network large enough to overfit, then regularize. This chapter is about the second half of that sentence.

Planned content: Slides not yet available. Expected structure below; adapt it to the slides.

8.1 Weight Decay (L_2 and L_1 Penalties)

Planned content: Adding a penalty to the empirical error; probabilistic interpretation as a prior; effect on separable data (Section 4.3).

8.2 Early Stopping

Planned content: Monitoring the validation loss (learning curves, Section 4.10); stopping criterion.

8.3 Dropout and Other Noise-Based Methods

Planned content: Dropout at training and test time; data augmentation; other methods as covered.

8.4 Chapter Checklist

Planned content: To be written together with the chapter.

9

Initialization

Section 4.10 gave the recipe "zero mean, variance $2/J$ for ReLU, $1/J$ otherwise". This chapter derives where these numbers come from.

Planned content: Slides not yet available. Expected structure below; adapt it to the slides.

9.1 Why Initialization Matters

Planned content: Symmetry breaking; connection to vanishing and exploding gradients (Section 4.7.3).

9.2 Variance-Preserving Initialization

Planned content: Propagation of mean and variance through a layer; LeCun/Glorot (Xavier) initialization; He initialization for ReLU; forward and backward view.

9.3 Chapter Checklist

Planned content: To be written together with the chapter.

10

Normalization

Normalizing the inputs speeds up training (Section 4.10). Normalization methods apply the same idea to the hidden layers, so that activations stay well-scaled throughout a deep network.

Planned content: Slides not yet available. Expected structure below; adapt it to the slides.

10.1 Input Normalization Revisited

Planned content: Standardization, whitening; why it helps optimization.

10.2 Batch Normalization and Layer Normalization

Planned content: Normalizing pre-activations with mini-batch or per-sample statistics; learnable scale and shift; behaviour at training versus test time.

10.3 Self-Normalizing Networks

Planned content: SELU activation (Table 4.2) and its fixed point of zero mean and unit variance; requirements on initialization.

10.4 Chapter Checklist

Planned content: To be written together with the chapter.